INTERNATIONAL JOURNAL OF GEOGRAPHICAL INFORMATION SCIENCE
2022, VOL. 36, NO. 9, 1853–1884
https://doi.org/10.1080/13658816.2022.2087222

Taylor & Francis
Taylor & Francis Group

RESEARCH ARTICLE

Check for updates

A multi-source spatio-temporal data cube for large-scale geospatial analysis

Fan Gao[a], Peng Yue[a,b,c], Zhipeng Cao[a], Shuaifeng Zhao[a], Boyi Shangguan[a], Liangcun Jiang[a], Lei Hu[a], Zhe Fang[a] and Zheheng Liang[d]

[a]School of Remote Sensing and Information Engineering, Wuhan University, Wuhan, Hubei, China; [b]Collaborative Innovation Center of Geospatial Technology, Wuhan, Hubei, China; [c]Hubei Province Engineering Center for Intelligent Geoprocessing (HPECIG), Wuhan University, Wuhan, Hubei, China; [d]South Digital Technology Co, Ltd., Guangzhou, Guangdong, China

ABSTRACT

Data management and analysis are challenging with big Earth observation (EO) data. Expanding upon the rising promises of data cubes for analysis-ready big EO data, we propose a new geospatial infrastructure layered over a data cube to facilitate big EO data management and analysis. Compared to previous work on data cubes, the proposed infrastructure, GeoCube, extends the capacity of data cubes to multi-source big vector and raster data. GeoCube is developed in terms of three major efforts: formalize cube dimensions for multi-source geospatial data, process geospatial data query along these dimensions, and organize cube data for high-performance geoprocessing. This strategy improves EO data cube management and keeps connections with the business intelligence cube, which provides supplementary information for EO data cube processing. The paper highlights the major efforts and key research contributions to online analytical processing for dimension formalization, distributed cube objects for tiles, and artificial intelligence enabled prediction of computational intensity for data cube processing. Case studies with data from Landsat, Gaofen, and OpenStreetMap demonstrate the capabilities and applicability of the proposed infrastructure.

ARTICLE HISTORY
Received 23 July 2021
Accepted 5 June 2022

KEYWORDS
Data cube; high-performance computing; earth observation; cloud computing; artificial intelligence

1. Introduction

Earth observation (EO) data, a prime source of big data, comprise large-scale and multi-source geospatial data that are acquired from orbital sensors, in-situ measurements, and simulation models (Baumann *et al*. 2018, Wagemann *et al*. 2018). The management of EO data is challenging owing to the demand for the integration of multiple data sources, high-performance processing, and diverse applications of EO data. These multi-source EO data are characterized by the difference in spatial, temporal, and spectral resolutions, varying conditions in observations (e.g. different atmospheric conditions and viewing geometries), and diverse spatial data structures (e.g.

CONTACT Peng Yue pyue@whu.edu.cn

© 2022 Informa UK Limited, trading as Taylor & Francis Group

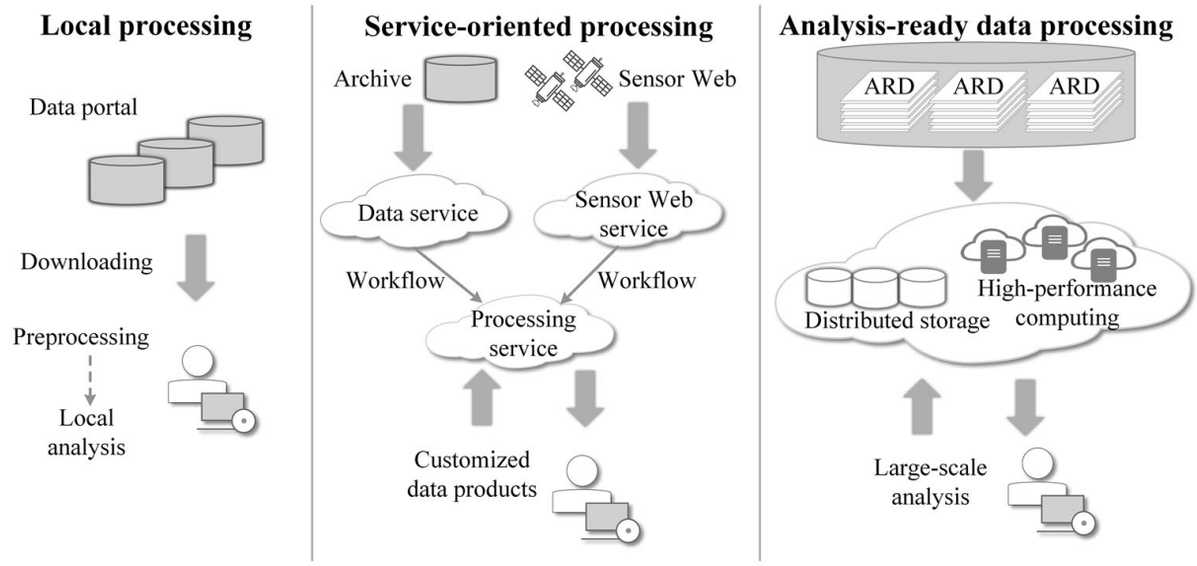


Figure 1. Paradigm shift in EO processing.

raster and vector data) (Lewis *et al*. 2016). These characteristics make EO data integration, storage, and processing a challenging topic.

Recent advances in information infrastructures have shown significant promises for EO data processing (Yue *et al*. 2016, Sudmanns *et al*. 2019) (Figure 1). Several paradigm shifts occurred in the EO domain between the 1990s and 2000s. In the local processing paradigm, EO data are often downloaded from data portals and processed in local machines. This may result in high network bandwidth resources and time costs. The capabilities of users are limited in solving large-scale geospatial problems owing to insufficient local storage space and computing power (Wulder and Coops 2014, Kopp *et al*. 2019). The development of Web technologies and geospatial Web services has made it possible for users to discover, access, and process geospatial data on spatial data infrastructure with ease (Zhao *et al*. 2012). Examples include the Geospatial Interactive Online Visualization and Analysis Infrastructure that was developed by the Goddard Earth Sciences Data and Information Services Center (Berrick *et al*. 2009), and Grid Processing on Demand System that was developed by European Space Agency (ESA 2012). Users can customize data products using scientific workflows that are composed of geospatial Web services such as data services, sensor Web services, and processing services. Geospatial data can be accessed via the Internet in an interoperable manner, and the processing can be migrated to remote servers instead of being performed on local machines (Yue *et al*. 2010). This is known as the service-oriented processing paradigm. In recent years, the analysis-ready pattern has emerged as a revolutionary paradigm for big geospatial data processing (Gorelick *et al*. 2017, CEOS 2021). It requires EO data to be preprocessed into an analysis-ready form, following which the Cyberinfrastructure is harnessed to make the data available to users. Analysis-ready data (ARD) means that the EO data are processed following a minimum set of requirements, thereby enabling immediate analysis with a minimal additional user effort and interoperability (Lewis *et al*. 2018). ARD can be incorporated into a geospatial data cube, which makes big EO data more accessible and enables

large-scale analysis with high-performance computing (HPC) technologies (e.g. parallel computing or cloud computing).

This study presents a cube infrastructure for big geospatial data analysis. In comparison with existing cube approaches, the infrastructure emphasizes the accommodation of multi-source geospatial data, including raster and vector data in the cube, cube tile processing with cloud computing, and artificial intelligence (AI) machine learning-enabled cube computation. AI-enabled cube computation refers to the use of AI for predicting the computational intensity (CI) of parallel cube processing tasks, which aids in improving the load balance performance. The design and implementation of the cube infrastructure, which is known as GeoCube, are detailed in this paper. It covers the ingestion and management of multi-source geospatial data in the cube, geospatial data querying and access along different cube dimensions, and high-performance cube computing of large-scale geospatial datasets. The development of such a geospatial data cube can contribute to advancing the EO data cube approach, as well as keep connections with the business intelligence (BI) cube.

The remainder of this paper is organized as follows: Section 2 introduces the background and related work. The proposed method is described in Section 3. The implementation and evaluation of GeoCube are detailed in Section 4. Section 5 presents a discussion and Section 6 concludes the study.

2. Background and related work

2.1. Data cube concept

The term 'data cube' originates from the BI domain. As a data cube generally refers to an array of multiple dimensions that facilitates online analytics, it is also known as an online analytical processing (OLAP) cube (Gray *et al*. 1997). The OLAP cube was introduced into the geospatial domain to accommodate spatial entities in vector data by adding the spatial dimension and spatial measure, thereby promoting OLAP to spatial OLAP (SOLAP) (Han *et al*. 1998). In recent years, this term has been used in the EO domain to refer to a time-series multi-dimensional array (Table 1). It has been incorporated into Cyberinfrastructure to create EO data cube infrastructure, which facilitates EO data management, access, analysis, visualization, and interoperability. Table 1 lists several typical definitions of data cube, covering definitions from the OLAP, SOLAP, array database (Baumann 2017), Earth Science Informatics (Nativi *et al*. 2017), and Open Geospatial Consortium (OGC) (Voidrot and Percivall 2020) communities. In the remainder of this paper, the former two are referred to as BI data cubes, whereas the latter three are categorized as EO data cubes.

It is often recognized that EO data cubes are not similar to BI data cubes, despite the cube concepts being similar. EO data cubes are always densely populated, whereas BI data cubes are typically sparse, which limits the use of the OLAP approach in EO data cubes (Strobl *et al*. 2017). A major challenge in bridging the two cubes is the manner in which the data are organized in high-performance forms for efficient cube processing. In a traditional BI data cube, a relational database is often selected for cube management, which is possible owing to the sparse datasets. However, EO

Table 1. Several definitions of data cube.

Reference	Definition
Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals (Gray *et al*. 1997)	An OLAP cube is a multi-dimensional array of data. The 0 D data cube is a point. The 1 D data cube is a line with a point. The 2 D data cube is a cross tab, a plane, two lines, and a point. The 3 D data cube is a cube with three intersecting 2 D cross tabs.
Selective materialization: An efficient method for spatial data cube construction (Han *et al*. 1998)	A spatial data cube is a subject-oriented, integrated, time-variant, and non-volatile collection of both spatial and nonspatial data, which consists of both spatial and nonspatial dimensions and measures.
The Datacube Manifesto (Baumann 2017)	A datacube is a massive multi-dimensional array, also called 'raster data' or 'gridded data'; 'massive' entails that we talk about sizes significantly beyond the main memory resources of the server hardware. Data values, all of the same data type, sit at grid points as defined by the d axes of the d-dimensional datacube. Coordinates along these axes allow addressing data values unambiguously. A d-dimensional grid is characterized by the fact that each inner grid point has exactly two neighbors along each direction; border grid points have just one.
A view-based model of data-cube to support big earth data systems interoperability (Nativi *et al*. 2017)	A time series multidimensional (space, time, data type) stack of spatially aligned pixel ready for analysis.
OGC Geospatial Coverages Data Cube Community Practice (Voidrot and Percivall 2020)	A multi-dimensional ('n-D') array of values. Even though it is called a 'cube', it can be 1-dimensional, 2-dimensional, 3-dimensional, or higher-dimensional. The dimensions may be coordinates or enumerations, e.g. categories. A geospatial data cube is a data cube established on basis of a coverage with a specific grid system for geospatial data with at least one dimension of spatial or temporal definition.

data cubes often require higher-performance solutions. Moreover, traditional SOLAP can accommodate vector data, whereas EO data cubes focus on raster data. It is possible to extend the spatial dimension and spatial measure into EO data cubes to enable cube analysis on multi-source datasets. Thus, it is necessary to investigate approaches for bridging EO and BI data cubes by augmenting cubes with multi-source geospatial data ingestion and management, and performing cube processing of joint data analysis between dense data (EO data) and sparse data (vector or tabular data). Therefore, in this study, the geospatial data cube is understood as a time-series multi-dimensional data model, in which multi-source geospatial data can be organized as spatially aligned ARD in a high-performance form. Multi-source geospatial data are not limited to EO data, such as remote sensing images, but also can be vector, trajectory, point cloud, or tabular data. The geospatial data cube makes big EO data more accessible and enables large-scale analysis using HPC technologies.

2.2. BI data cube

The BI data cube was first introduced by Gray *et al*. (1997) for business data management, and it represents business data in the form of a multi-dimensional array using concepts such as the dimension, measure, cell, and level. A data cube is composed of cells, which can be defined based on their dimensions and measures. The dimensions

represent the analysis axes and a measure is a numerical variable that is identified by a combination of dimensions. A dimension can be organized into levels hierarchically, with each level corresponding to a different granularity for that dimension (Damiani and Spaccapietra 2008). Considering a marketing sales example, a set of dimensions can be defined, such as the location, time, and product type. The measure can be defined by the values of the sales. Thus, it can be identified using the location, time, and product type dimensions. Among these dimensions, the time dimension can be represented by different levels, such as day, month, and year, so that the values of the sales can be aggregated along the dimension (Rivest *et al*. 2001).

EX. OF BUSINESS D.CUBE

A BI data cube is always used with the OLAP approach. OLAP can store different cube versions of a dataset at different aggregation levels, thereby providing complex and fast aggregation processes to summarize the data (Kasprzyk and Devillet 2021). OLAP defines several operations for users to explore a data cube (Chaudhuri and Dayal 1997):

- trim and slice, which performs data selection along dimensions;
- roll up, which moves up along one or more dimensions towards more aggregated data;
- drill down, which moves down along one or more dimensions towards more detailed data; and
- pivot, which reorients the data cube for a different view.

Spatial attribute is an important dimension of geospatial data. The straightforward use of an OLAP dimension as a spatial dimension cannot support the aggregation processes of spatial entities. Therefore, SOLAP was proposed, which defines the meanings of the spatial dimension and spatial measure (Han *et al*. 1998). The spatial dimension is associated with a geometric entity, which may have different spatial granularities, such as country, province, and city. A spatial measure is a collection of pointers for the spatial entities (Han *et al*. 1998, Bimonte *et al*. 2010). Thus, the spatial dimension is similar to a spatial index, whereas the spatial measure is similar to a collection of spatial features. Such a model makes it possible to perform spatial aggregation functions, such as intersection and union, by means of which a set of geometries can be returned from the spatial measure (Silva *et al*. 2008).

OLAP provides multiple methods for implementing data cubes in databases. The most commonly used methods are the star, snowflake, and fact constellation schemas (Chaudhuri and Dayal 1997). In the star schema, the data cube is represented as a star, the center of the star is a fact (i.e. measures in cube cells) table, and the points surrounding the star consist of dimension tables (Kimball and Ross 2011). Although the star schema is the simplest implementation of the data cube model in a database, it cannot explicitly support dimension hierarchies. Therefore, the snowflake schema was developed to refine the star schema. In the snowflake schema, the dimension hierarchy can be represented by normalizing the dimension tables. The fact constellation schema is more flexible because it allows multiple fact tables to share the dimension tables in a database (Chaudhuri and Dayal 1997). In this manner, it provides the

Table 2. Comparison of different data cubes.

	EarthServer data cube	Open data cube	Earth system data cube	GeoCube
Data source	Raster data	Raster data	Raster data	Raster data/ vector data
Spatial OLAP	No	No	No	Yes
Computing pattern	HPC	HPC	HPC	HPC/AI-enabled computational intensity prediction
Data organization	Raster tile	Raster tile	Raster tile	Raster tile/vector tile
Metadata	PostgreSQL/SQLite (GMLCOV model)	PostgreSQL (JSONB)	File system (config file)	PostgreSQL (fact constellation schema)
Array-based interface provided	Yes	Yes	Yes	Yes
Data indexing	Tile indexing	Scene and tile indexing	Tile indexing	Tile indexing
Data ingestion	Mandatory	Optional	Mandatory	Mandatory
User interaction	WCS/WCPS, rasql, C++/Java/R/ JavaScript API	WCS/WMS/WPS, Jupyter Notebook, Python API	FTP/WCS, Jupyter Notebook, Python/Julia/R API	OGC API – Coverages/ Processes, OGC Data Access and Processing API, Jupyter Notebook, Scala/Python API

possibility of implementing a data cube consisting of multi-source and heterogeneous geospatial data that share several common dimensions.

2.3. EO data cube

Data cubes have gained increasing attentions in recent years and have exhibited great promise in the EO domain for extracting useful information from big EO data (Baumann *et al*. 2016, Giuliani *et al*. 2017, Sudmanns *et al*. 2019, Schramm *et al*. 2021). An EO data cube can be defined as a time-series multi-dimensional (space, time, and data type) stack of spatially aligned pixels that are ready for analysis (Nativi *et al*. 2017). Although existing EO data cubes focus on images, in this study, the cube concept is extended to cover other geospatial data sources, such as vector data, thereby providing a multi-source geospatial data cube.

Table 2 presents a comparison of three typical EO data cubes and the proposed GeoCube. The EarthServer data cube was created using the Rasdaman Array Database system (Baumann *et al*. 1999). It supports intercontinental-scale data management, ad-hoc extraction, and processing (Baumann *et al*. 2016, 2018). The cube can use either the PostgreSQL database or file systems to store data, although the PostgreSQL option has recently been deprecated. The metadata are represented using the OGC Geography Markup Language Coverages (GMLCOV) specification, which is stored in PostgreSQL or SQLite. Multiple data indexing methods are supported, such as the R + tree and regular computed index. The EarthServer data cube supports the Web service interface that is provided by the Petascope on top of Rasdaman, including the OGC Web Coverage Service (WCS) and Web Coverage Processing Service (WCPS).

These service requests can be translated into the Rasdaman Query Language (RaSQL) and executed in the Rasdaman database (Rasdaman 2021).

The Open Data Cube (ODC) initiative is an open-source project that was launched by the Committee on Earth Observation Satellites (CEOS) (CEOS 2021, Gomes *et al*. 2021). The objective of the ODC is to lower technical barriers and to provide a solution for national or local agencies to create data cubes that are similar to the Australian geoscience data cube, Colombia data cube, and Swiss data cube (Lewis *et al*. 2017, CEOS 2021). The ODC can manage large amounts of satellite data and process national-scale or even continent-scale satellite data using HPC technologies. It allows to store data in the original format or ingest data in a high-performance tile format. The former adopts the traditional scene-based index, whereas the latter uses tile-based index. The same API can be enforced on top of the two organizing forms, although the former option was recently recommended for the ODC. The Dask parallel computing engine can be used in the ODC for parallel Earth science, satellite imagery, and time-series analyses (Dask 2021). Unlike the Dask in the ODC, Apache Spark (Zaharia *et al*. 2010) is selected for GeoCube, because it has a wider user base in open-source cloud computing communities, with more programming languages and a comprehensive software ecosystem covering Hadoop, HBase, and Geotrellis. In the ODC, the EO data in the tile format are stored using the Network Common Data Form (NetCDF). The metadata are encoded using the JSONB format and managed in the PostgreSQL database (Lewis *et al*. 2017). Five relational tables are leveraged to manage the cube data. Furthermore, the ODC provides the Web service interface, including the OGC WCS, Web Map Service (WMS), and Web Processing Service (WPS) (Voidrot and Percivall 2020).

The objective of the Earth System Data Cube (ESDC) is to explore various EO raster data streams simultaneously to enable joint analysis. Multiple data streams are integrated into a single high-dimensional data cube. The infrastructure provides the same data cube at different spatial resolutions and chunkings to accelerate data access for different applications (Mahecha *et al*. 2020). The cube data are encoded using the Zarr format (Zarr 2021), which is a format that is optimized for cloud storage. The ESDC uses a file system directory to organize the data, and each cube instance has a configuration file for describing the spatial and temporal resolution. The ESDC enables data retrieval and downloading via file transfer protocol (FTP). Moreover, the OGC WCS and Jupyter Notebook are supported for data access and analytics (ESDC 2021).

2.4. Contributions of the work

This study presents a new cube infrastructure for big geospatial data analysis. The method differs from existing cube approaches in the following aspects: (1) The infrastructure supports the accommodation of multi-source geospatial data, including raster and vector data in the cube. The multi-source data are segmented into a tile form in a NoSQL database, ready for HPC, thereby enabling the user to perform efficient multi-source data analysis using the high-performance tile form. (2) The infrastructure supports large-scale and joint analysis for cube tiles using a cloud-based cube object. It is compatible with multiple data sources, which can help solving the problem of the

joint analysis of multi-source and heterogeneous geospatial data. SOLAP and EO data cubes work together in a cloud computing environment. Cloud computing refers to the on-demand use of storage and computing resources that are located on networked computers. Existing cloud computing technologies such as Apache Spark have been used extensively for big data processing. Apache Spark is a high-performance cloud computing engine that was developed for data-intensive applications. It incorporates a data model known as the Resilient Distributed Dataset (RDD), which is a data collection that is partitioned across a set of machines for distributed computing (Zaharia *et al*. 2010). The leveraging of cloud computing and this data model can significantly improve the processing capabilities of the cube infrastructure. (3) The infrastructure improves the load balance of parallel geoprocessing using AI. Although cloud computing empowers the infrastructure with parallel geoprocessing capabilities, geospatial data are heterogeneous and spatially distributed in an irregular manner, which often leads to a load imbalance owing to the various computational intensities of the different tiles. AI machine learning technology, which has shown potential for the evaluation of CI (Yue *et al*. 2020), can be incorporated into the distributed tile processing to improve the load balance.

WE USE CYVERSE

3. Materials and methods

This section describes the approach for constructing the multi-source spatio-temporal data cube for large-scale geospatial analysis. The approach includes the formalization of cube dimensions to accommodate multi-source data (Section 3.1), cube tile processing using cloud computing (Section 3.2), and enhanced cube computation using AI (Section 3.3).

A new geospatial data cube with the following characteristics is proposed. First, multi-source geospatial data, including raster and vector data, can be accommodated in the cube. Four dimensions, namely time, space, product, and band, are formalized. The multi-source data are spatially aligned and stacked in a time-series form. Thus, it is named as a multi-source spatio-temporal data cube. The cube is developed using the OLAP fact constellation schema, so that OLAP operations such as spatial merge, map algebra and time-series analysis can be performed on the cube (Section 3.1). Second, cloud computing is used for processing the cube tiles. Distributed cube objects for cube tiles are proposed to facilitate the distributed computing (Section 3.2). Third, AI technologies are applied to enhance the load balance in the cube computation. The heterogeneity of the geospatial data can be captured from a data science perspective using AI machine learning, thereby optimizing the parallel computing performance (Section 3.3).

3.1. Cube dimension formalization

The cube dimensions are important for the development of a cube data model. Geospatial data can be represented in multiple dimensions, such as the space, time, and thematic properties. The model specifies several basic dimensions to accommodate the raster, vector, and tabular data, as illustrated in Figure 2.

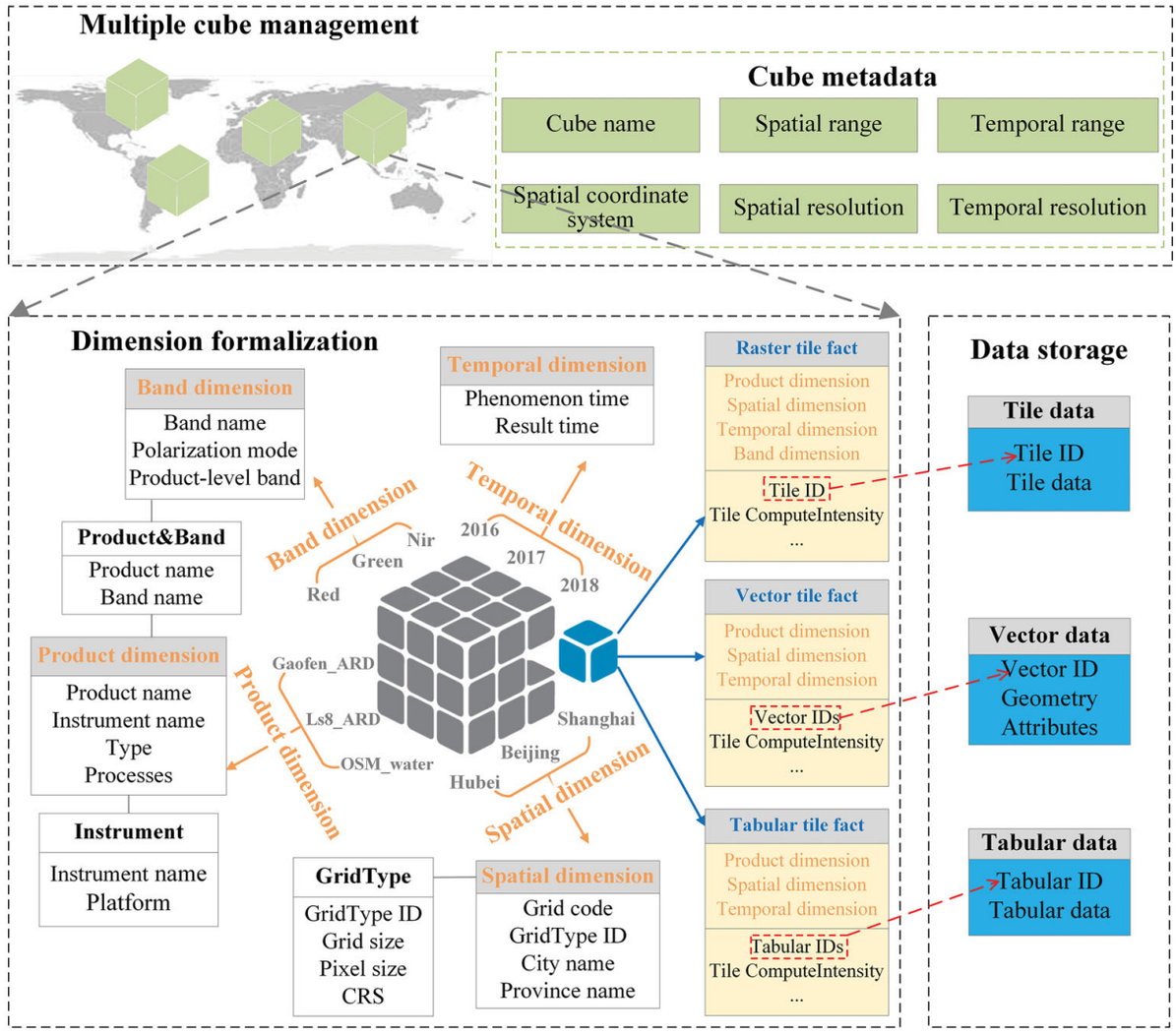


Figure 2. Cube data model. ADAPT TO MY RESEARCH FRAMEWORK

3.1.1. Product dimension

This dimension specifies the thematic axis for the geospatial data cube using the product name (e.g. Landsat_8_ARD or OSM_Water), type (e.g. raster, vector or tabular), processes, and instrument name. The product and instrument name, are linked to the Product&Band and Instrument tables, respectively, which are used as supplementary tables when consuming raster data. For example, the product dimension can describe optical and SAR image products by recording information on the instrument and band. The supplementary tables are not necessary when vector data products such as OpenStreetMap (OSM) are considered. The processes are the functions that have been supported and can be performed on the product. Data querying or subsetting can be performed along the product dimension using the product name.

3.1.2. Spatial dimension

This dimension specifies the spatial axis for the geospatial data cube using the grid code, grid type, city name, and province name. The cube uses a spatial grid for tiling to enable data readiness in a high-performance form. The tiling for raster data involves dividing the data into physical tiles, whereas the tiling for vector data involves indexing the data as logical tiles instead of a physical division, which aids in maintaining

the completeness of the vector features. The resolution of the spatial grid is customizable, e.g. a 1×1 grid tile (grid size) that consists of 4000×4000 pixels (pixel size). The grid type table is used for recoding the layout of a grid, including the grid and pixel sizes, as well as the coordinate reference system (CRS). The CRS may be the World Geodetic System 1984 (WGS 84) or a projected coordinate reference system such as the Universal Transverse Mercator, depending on user specification. A grid code is assigned to each tile to enable the indexing of the grid tiles. The code follows Z-order curve-based encoding so that contiguous tiles are assigned to the same node, and thus, the query and computing performance can be improved owing to the reduced communication cost. When a bounding box query is considered, the grids with which the bounding box intersects will be determined first, following which the grids that are encoded using the Z-order curve can be found in the database. The tile data that are identified by the grid codes are often located in the same node. The city and province names can be used to support certain OLAP aggregation operations.

3.1.3. Temporal dimension

This dimension specifies the temporal axis for the geospatial data using the phenomenon time and result time. The phenomenon time is the time when the observation is captured such as the acquisition time for satellite imagery and the update time for vector and tabular data, whereas the result time is the time when the result becomes available once all processing steps have been completed. Data querying or subsetting along the temporal dimension can be performed using the phenomenon time.

3.1.4. Band dimension

This dimension describes the band attribute of the raster products according to the band name, polarization mode that is reserved for SAR images, and product-level band. The product-level band is the information that is extracted from the original bands. For example, the Normalized Difference Water Index (NDWI) band is a product-level band that is generated from original green and near-infrared bands.

The model allows one or multiple cubes to be created based on user specifications. Each cube has a uniform spatial resolution and a CRS. Multiple cubes can be created for multiple datasets, which aids in preserving the original information. It is possible to perform queries across different cubes, and subsequently to re-sample and re-project different cube data into a uniform spatial resolution and datum. Furthermore, a cube in which multi-source datasets are spatially aligned can be created. This option requires the re-sampling and re-projection of the dataset into a uniform spatial resolution and datum. The cube management part in Figure 2 presents the cube metadata schema.

The dimension formalization part in Figure 2 follows a fact constellation schema. The raster, vector, and tabular tile fact tables share product, spatial, and temporal dimensions. The tabular data have the minimum requirement of the spatial dimension being ingested in a spatially aligned cube. This can be achieved by a place name attribute in the table data, which can subsequently be transformed into the spatial dimension using existing geocoding services. Two types of measures are used in the cube data model: data and computational measures. The data measure is a reference to a

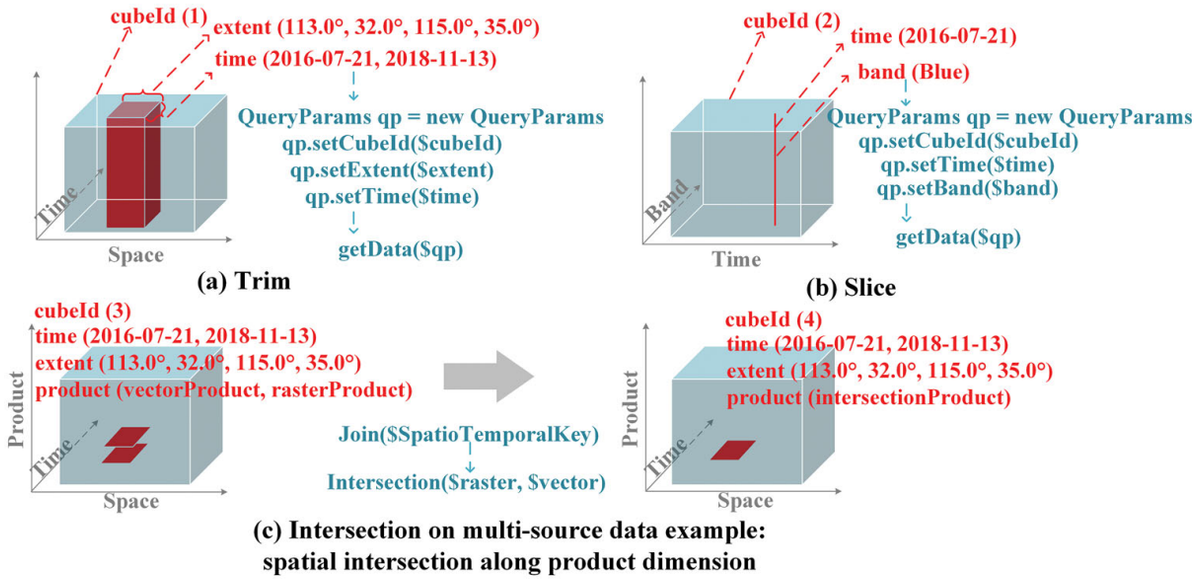


Figure 3. Cube OLAP operations.

tile, which uses the tile ID for the physical raster tile, vector IDs for the features that spatially intersect with a logical vector tile, or tabular IDs for the tabular records that spatially intersect with a logical tabular tile. The computational measure consists of values that are evaluated based on the computational intensity of the tiles. The following sections describe how to take advantage of the cube data model.

The formalization of dimensions in the cube data model helps to support OLAP operations on geospatial data, such as trim, slice, and roll up. Trim and slice are the simplest operations that are used for geospatial data in a cube (Baumann 2017, Voidrot and Percivall 2020). Trim selects multiple values along varying dimensions, whereas slice can produce a sub-cube of reduced dimensions by selecting a single value along a dimension. Figure 3(a) depicts a trim operation along the temporal and spatial dimensions, which can return a stack of tiles within a given spatial extent and time. A slice operation along the band and temporal dimensions is illustrated in Figure 3(b). Several interfaces for querying cube data have been presented, such as *getData($QueryParams),* which is used to access raster/vector/tabular data. The *QueryParams* class can be initiated using a cube identifier and a set of dimensional information.

Roll up is a more complex OLAP operation, it is a data aggregation process along different dimension levels, such as the grid/city/province in the spatial dimension, day/month/year in the temporal dimension, original/product-level band in the band dimension, and different-level products in the product dimension. The results of the OLAP operation can be pre-computed and saved to create a new cube of different dimension levels, leading to a faster response from the user. Considering a flood disaster example, intersection analysis is performed between inundation images (a type of raster data) and villages (a type of vector data) to identify flooded villages (Figure 3(c)). Aggregation occurs along the product dimension, the flooded village is a higher-level product, which can help to create a new cube with a higher-level product dimension. The aggregation involves the *Join* and *Intersection* operations. The *Join* operation is used to join the multi-source data based on the spatial and temporal

identifiers, thereafter, the *Intersection* operation can be performed between the raster and vector data in the same spatial and temporal space. This example may involve more roll up aggregations, such as the spatial merge function for tile mosaics along the spatial dimension and the sum function for the total affected population calculation along the spatial dimension.

3.2. Cube tile processing using cloud computing

It is important to understand how data are organized in high-performance forms for efficient cube processing. This section explains the leveraging of cloud computing to design distributed cube objects for tiles, thereby facilitating high-performance distributed computing.

3.2.1. Distributed cube objects for tiles. Distributed cube objects for tiles are designed by bridging tiles with distributed objects in cloud computing. The raster data are segmented into tiles with a uniform projection and resolution in terms of the spatial dimension in the cube. The vector data are segmented into logical tiles to maintain their topology. A logical tile means that the vector data are not physically segmented yet attached to a grid if the vector intersects with the grid. This can help to preserve the geometric information of the vector data and facilitate their parallel spatial analysis. The data tiles are organized at certain time intervals, such as a day, week, month, and year in the temporal dimension, whereas the tiles are organized according to the classification of the product/band types in the product/band dimension. Therefore, the raster tiles can be described by the product/band, spatial, and temporal dimensions, whereas vector tiles can be described using the product, spatial, and temporal dimensions.

Typical software, such as Apache Spark, includes distributed objects for HPC in cloud computing. The core object model is the Spark RDD class. Thus, it is possible to combine the distributed computing capability of the Spark RDD with the tiles in the cube, which results in data parallelism in HPC. Subsequently, a set of distributed cube objects is employed for the cube tiles. Figure 4 presents the hierarchy and transformations of the objects. CubeRDD[T] is the top class that contains a collection of key-value pairs, which are inherited from the abstract RDD class. The key type of CubeRDD[T] is fixed (i.e. the SpaceTimeProductKey class), and involves the product, spatial and temporal dimensions, which are common dimensions for multi-source geospatial data, including raster and vector data. The value type T is a geospatial data type (e.g. raster and vector tile). It is designed to support multi-source data, thereby making the cube objects extensible, which results in the accommodation of additional data types such as trajectory data. CubeRDD[T] also provides grid layouts, spatial and temporal extents, and CRS information.

The RasterRDD for raster data and FeatureRDD for vector data are designed based on CubeRDD[T]. A RasterRDD can be transformed from a CubeRDD[RasterTile], where the RasterTile class contains band information and an array of pixel data. Thus, RasterRDD can contain the band dimension in addition to the product, spatial, and temporal dimensions. A FeatureRDD can be transformed from a CubeRDD[FeatureCollection],

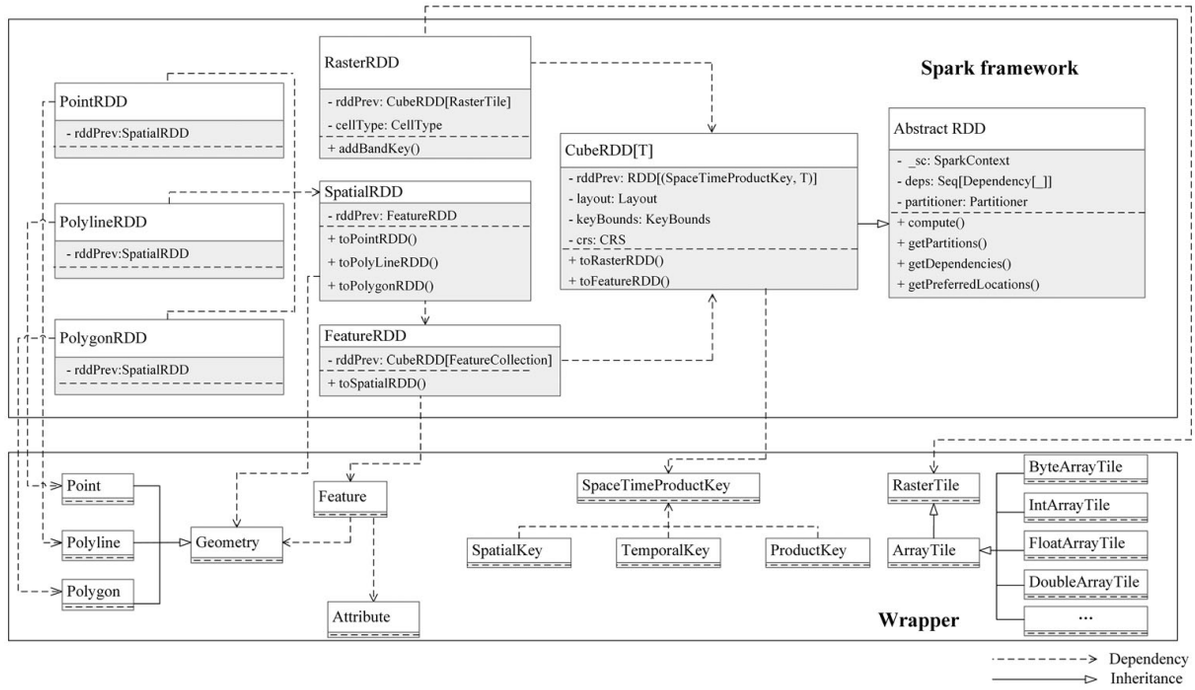


Figure 4. Distributed cube objects.

where the FeatureCollection class contains a set of geometries and their corresponding attributes. A FeatureRDD can be converted into a SpatialRDD by retaining only the geometric information. Subsequently, a PointRDD, PolylineRDD, and PolygonRDD can be created from a SpatialRDD because the geometry may be a point, polyline, or polygon.

The CubeRDD design makes it easier to perform geoprocessing along different dimensions. For example, a RasterRDD can be grouped along the temporal dimension to enable distributed time-series analyses or along the band dimension to perform distributed band algebra operations, such as the Normalized Difference Vegetation Index. A FeatureRDD can be grouped along the product dimension to support the distributed spatial intersection between two products. RasterRDDs and FeatureRDDs can be joined together to locate raster and vector tiles that share common dimensions, thereby enabling distributed joint analysis for multi-source geospatial data.

3.2.2. Cube querying and processing

The cube querying and processing operations are depicted in Figure 5. First, the cube receives and parses multi-dimensional query parameters, following which the tile IDs are retrieved from the dimension and fact tables. These tile IDs are partitioned and assigned to each computing unit to enable parallel data access. Cube processing includes batch processing, cloud computing, and multi-thread computing. The use of cloud computing and multi-thread computing forms hybrid parallelism, which refers to a blend of distributed and shared memory parallelism (LLNL 2021).

Batch processing solves the issue whereby a large number of tiles cannot be loaded into memory simultaneously, and it is used to divide the tiles into several batches. Each batch can be loaded into memory as a whole. Thus, batch processing enables a large number of tiles to be processed with limited memory. Each batch is accessed as a distributed cube object (CubeRDD). Furthermore, a multi-thread pool is created to process each single large tile. The size of a tile may be very large, for example, a high-

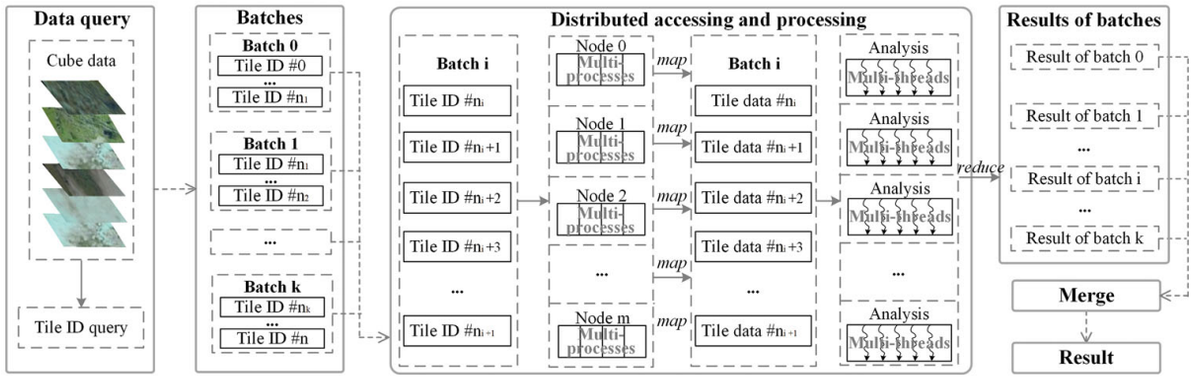


Figure 5. Cube processing using batch computing, cloud computing, and multi-thread computing.

resolution image or a grid containing a large number of polygons, which results in a high time cost in data processing. Thus, hybrid parallel processing is required for the cube processing. Each sub-thread processes only a portion of a tile in the multi-thread pool and the processing results are gathered in the main thread. Thus, data processing involves multiple processes among the computing nodes and multiple threads in one node, thereby forming hybrid parallelism.

For example, the time period 2017.01.01 to 2020.12.31 can be divided into 2017.01.01 to 2018.12.31 and 2019.01.01 to 2020.12.31, whereas the spatial extent (110°E, 25°N, 120°E, 35°N) can be divided into (110°E, 25°N, 115°E, 35°N) and (115°E, 25°N, 120°E, 35°N). Subsequently, the batch tiles that are grouped according to the aforementioned divisions are accessed and returned as distributed cube objects, which are processed in the Spark cloud environment. A multi-thread pool containing eight threads can be created in each process, and a tile is divided into eight sub-tiles for each thread. The processing results of the threads are gathered, and finally, the results of the batches are merged along the dimensions.

3.3. Cube computing optimization using AI

The data are organized as tiles that are loaded into distributed computing nodes in the cube. Different tiles have different CIs in terms of the geoprocessing algorithms and heterogeneous spatial feature distributions. Load balancing is a challenge in cube computation, and two methods are commonly used to address this issue: static and dynamic scheduling. Dynamic scheduling alleviates the load imbalance by monitoring the status of the clusters and assigning tasks to idle computing units. However, the advantages of this strategy are limited when the number of tasks is extremely low or high. Too few tasks may make it difficult to achieve load balance, whereas too many tasks often result in high communication cost. Static scheduling can divide and group data into partitions with balanced computational intensity in advance. The accuracy of the CI assessment is key to the success of this strategy.

GeoCube evaluates the CI of the tiles using AI machine learning approaches. This work is a continuation of the previous research on the machine learning approach for the prediction of the CI in parallel geoprocessing (Yue *et al.* 2020), which is extended to the evaluation of the tile CI. Figure 6 shows how the approach is used in the cube infrastructure to predict the CI of the cube tiles. A CI evaluation model for geospatial

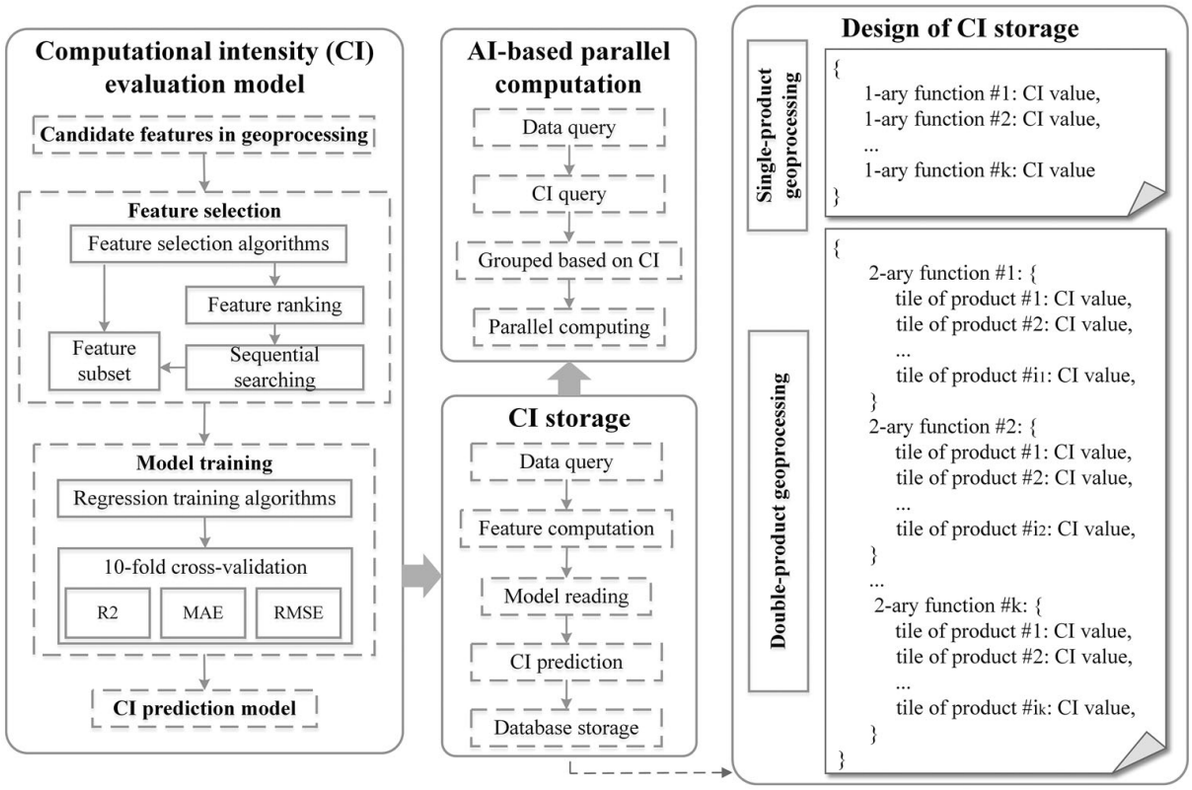


Figure 6. Cube computation using AI-enabled CI prediction approach.

analysis is generated using AI machine learning feature selection and regression training. Feature selection is employed to evaluate the importance of a set of candidate computational features that potentially affect the intensity. These features can be prepared based on empirical knowledge and assessed during training stage. The machine learning regression algorithms are evaluated using model training and validation to determine the best model for the CI prediction. The CI of the tiles can be evaluated using the model prior to the computation and saved in the cube infrastructure. When specific cube analyses are performed, the computing cost can be directly derived from the archived CI instead of evaluated on the fly, thereby enriching the ARD with more content and saving time in the analysis stage.

The enrichment of the CI with the cube follows a n-ary model corresponding to n-ary geoprocessing functions. In the 1-ary case, each tile has a single CI value corresponding to a 1-ary geoprocessing function with one input, such as triangulation and NDWI, which can be formalized as follows:

$$CI_k(t_{\{dimensions\}}) = f_k\left(\mathcal{X}(t_{\{dimensions\}})\right), \tag{1}$$

where $t_{\{dimensions\}}$ is a cube tile that is identified by the specific dimensions, $CI_k(*)$ is the CI value of the tile in the geoprocessing k, $\mathcal{X}(*)$ returns a list of computational features of the tile, f_k is a pretrained machine learning model for geoprocessing k, and f_k uses a set of computational features as inputs and returns a CI value.

In the 2-ary geoprocessing case, in which the function has two inputs such as a two-layer intersection, each tile has a CI value that is associated with the tile of another product in the cube. It can be formalized as follows:

$$CI_{k;i}(t_{\{dimensions\}}) = f_k\left(\mathcal{X}(t_{\{dimensions\}}, t_{i\{dimensions\}})\right), \tag{2}$$

where $CI_{k;i}(*)$ is the CI value of the tile that is associated with the tile of target product i in geoprocessing k, $t_{i\{dimensions\}}$ is the tile of target product i, and $\mathcal{X}(*)$ returns a list of computational features of the two tiles.

In the n-ary geoprocessing case, each tile has a CI value that is associated with the tiles of the other n-1 products in the cube, which can be formalized as follows:

$$\begin{aligned} &CI_{k;L}(t_{\{dimensions\}}) = f_k\left(\mathcal{X}(t_{\{dimensions\}}, t_{l_1\{dimensions\}}, t_{l_2\{dimensions\}}, \ldots, t_{l_{n-1}\{dimensions\}})\right), \\ &l_1, l_2, \ldots, l_{n-1} \in L, \end{aligned} \tag{3}$$

where $CI_{k;L}(*)$ is the CI value of the tile that is associated with the tiles of the other n-1 products L in geoprocessing k, $t_{L\{dimensions\}}$ is the associated tile list, and $\mathcal{X}(*)$ returns a list of computational features of the n tiles.

The aforementioned model provides a theoretical basis for setting the CI values in the cube. In practical applications, many geoprocessing functions are 1-ary or 2-ary cases. The JSON data structure can be used to record the CI values. More specifically, the CI values are saved in the fact table from Section 3.1 using JSON files, examples of which are depicted in Figure 6. One is for the geoprocessing on a single product, such as spatial buffering, and the other is for geoprocessing between two products, such as spatial intersection. In the former, each tile has a single CI value for a specific 1-ary geoprocessing function. In the latter, each tile may have a CI value that is associated with one of the target products in the cube for a specific 2-ary geoprocessing function, under the premise that the target products are semantically acceptable as the second input for the 2-ary geoprocessing function. For example, a tile may have CI values relating to the tiles of products (#1, #2, …) in a 2-ary function.

The tiles can be assigned to computing units for balanced processing based on the AI-predicted CI. The CI values are obtained when accessing the cube tiles, which can be grouped and assigned to computing units based on the CI values. Subsequently, an effective load balance can be achieved in the cube computation.

4. Implementation and evaluation

4.1. Implementation

The cube infrastructure was implemented as the GeoCube software. Figure 7 presents the GeoCube implementation architecture. Three types of interfaces are offered to users at the front end: Jupyter Notebook, the OGC geospatial Web service, and Web UI. The back end includes four modules: the data source, data storage, data access, and computing engine modules. The data source module manages multi-source geospatial data, which can be either EO data, such as Landsat, Gaofen, and Sentinel, vector data, such as OSM, or high-level remote sensing products.

Data from external sources can be downloaded and preprocessed into an analysis-ready level. A metadata database is available for managing and indexing the ARD. Users can retrieve the ARD based on the metadata and subsequently ingest the data into a cube. During the ingestion, the data are re-projected, re-sampled, and divided into tiles according to the cube configuration. The cube data storage module manages

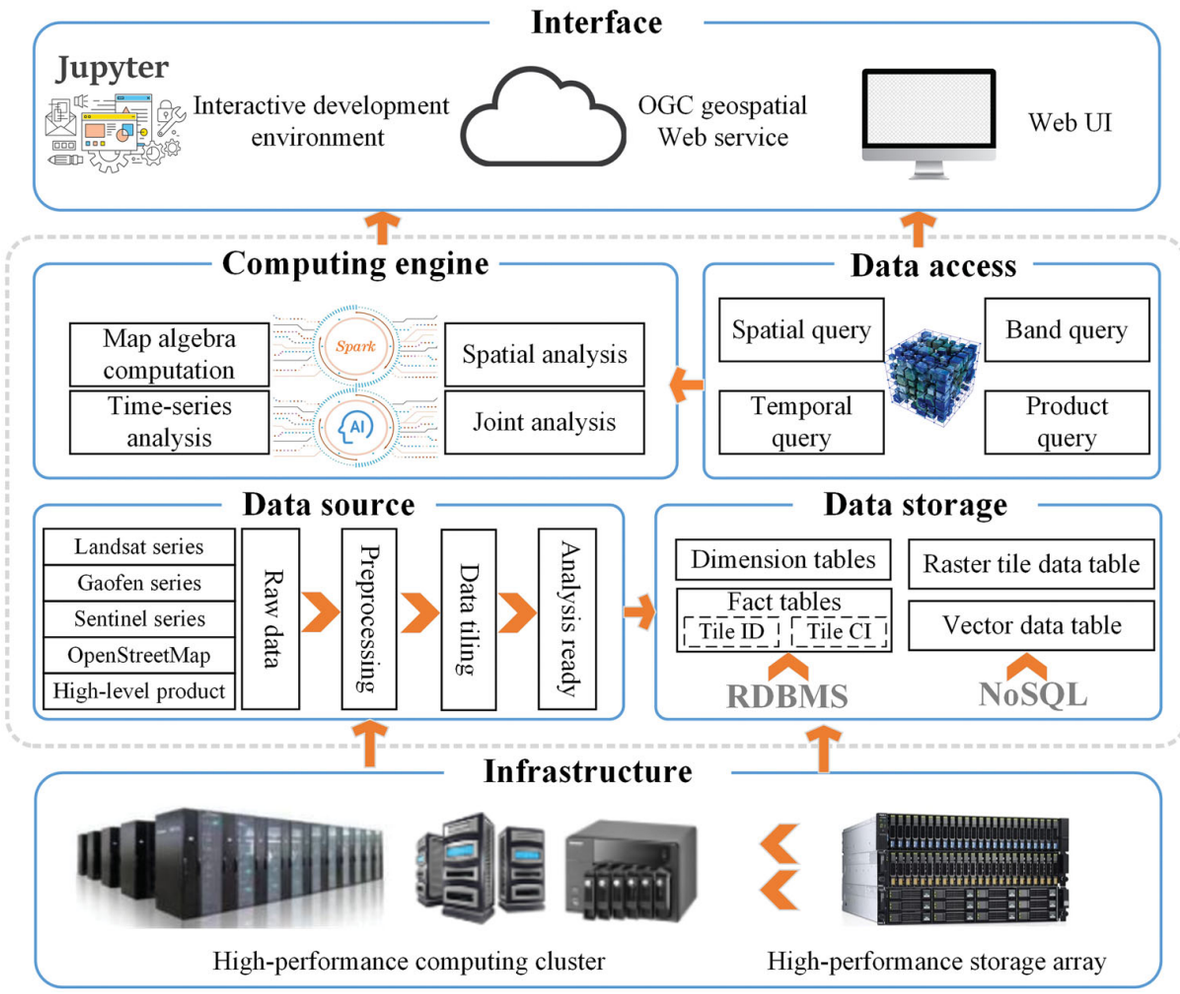


Figure 7. Implementation architecture of GeoCube.

the tiles by leveraging both the relational and NoSQL databases: the dimension and fact tables are managed using relational databases, whereas the raster and vector data tables are managed using distributed NoSQL databases. The raster data table records the tile ID and tile data bytes. The vector data table records the vector data IDs, geometric information that is encoded using well-known text (WKT), and attributes that are encoded using JSON. The data access module supports multi-dimensional queries for geospatial data along the spatial, temporal, product, and band dimensions.

The computing engine module integrates cloud computing with the distributed cube objects and AI-enabled CI prediction method to improve the computing performance of the cube data processing, such as the map algebra, time-series analysis, spatial analysis, and multi-source joint analysis.

In GeoCube, the fact constellation schema is implemented using PostgreSQL 9.3.24 (Momjian 2001), and the raster and vector tiles are saved in HBase 1.4.13 (George 2011), which is built on Hadoop 2.7.4 (White 2012). Spark 2.4.3 (Zaharia *et al.* 2010) and Geotrellis 3.0.0 (Geotrellis 2021) are used to develop the distributed memory cube objects, which are used for the parallel data ingestion, data access, and data processing. The machine learning algorithms are implemented using Python 3.7 and Scikit-learn 0.20.3 (Pedregosa *et al.* 2011). The OGC Web services are implemented using Springboot 2.3.1 (Spring Boot 2021). The standard service interface allows users to access the cube data and perform cube functions in an interoperable manner. For

example, users can access the standard coverage data instead of the cube tiles by using the OGC Coverages API. The Web-based interactive development environment is created using Jupyter Notebook (Jupyter 2021), and Cesium 1.71 is used for the data visualization (Cesium 2021).

4.2. Proof of concept

4.2.1. Cube OLAP operations

GeoCube enables users to navigate among the levels of the cube data, ranging from the most detailed to the most aggregated. For example, 'year' is a more aggregated level than 'month' and 'day' in the temporal dimension. This section provides an example of how to explore the cube data at two different levels. Figure 8(a) presents the NDWI analysis using the most detailed levels of spatial, temporal, and band dimensions. The level requests users to provide the spatial extent in the form of longitude/latitude (WGS 84), the time range that is specific to the 'day', and the original sensor bands. The spatial extent is transformed into a series of Z-order curve codes of covered grids. As designed in the cube data model, the grid is the most detailed level of the spatial dimension. Similarly, the day and original sensor band are the most detailed levels of the temporal and band dimensions, respectively.

Figure 8(b) depicts the NDWI analysis at a more aggregated level. Users can provide a city or province name in the spatial dimension. A city or province covers a number of grids that are more aggregated than the grid. The data that are covered by the city 'Wuhan' are used to derive the NDWI product in Figure 8(b). Query parameters that are more aggregated than the day, such as the month and year, are supported in the temporal dimension. The cube data with the same month or year can be aggregated. Data from 2017 are used to derive the NDWI product in Figure 8(b). The product-level band is more aggregated than the original band in the band dimension. For example, the NDWI band is more aggregated than the original bands, including green and near-infrared bands. The two bands can be aggregated to perform the map algebra operation to calculate the NDWI band.

The results that are derived from the aggregated-level analysis can be pre-computed and saved to create a new cube, which is defined by more aggregated-level dimensions. For example, the spatial dimension comprises cities such as Wuhan instead of grids, the temporal dimension consists of years such as 2017 instead of days, and the band dimension comprises product-level bands such as NDWI instead of the original bands. The precomputing approach can lead to a faster response to users by querying the new cube instead of performing computation on the fly.

4.2.2. Large-scale time-series geoprocessing

A significant benefit of the cube is that it supports large-scale time-series geoprocessing. We use the computation of the water presence frequency analysis as an example. It is a time-series analysis that detects the presence of water during a period and summarizes the results as the number of times that water is detected at each pixel. The analysis involves spatial merging (i.e. tile mosaics) and temporal aggregation (i.e. pixel value averaging) (Mueller *et al*. 2016).

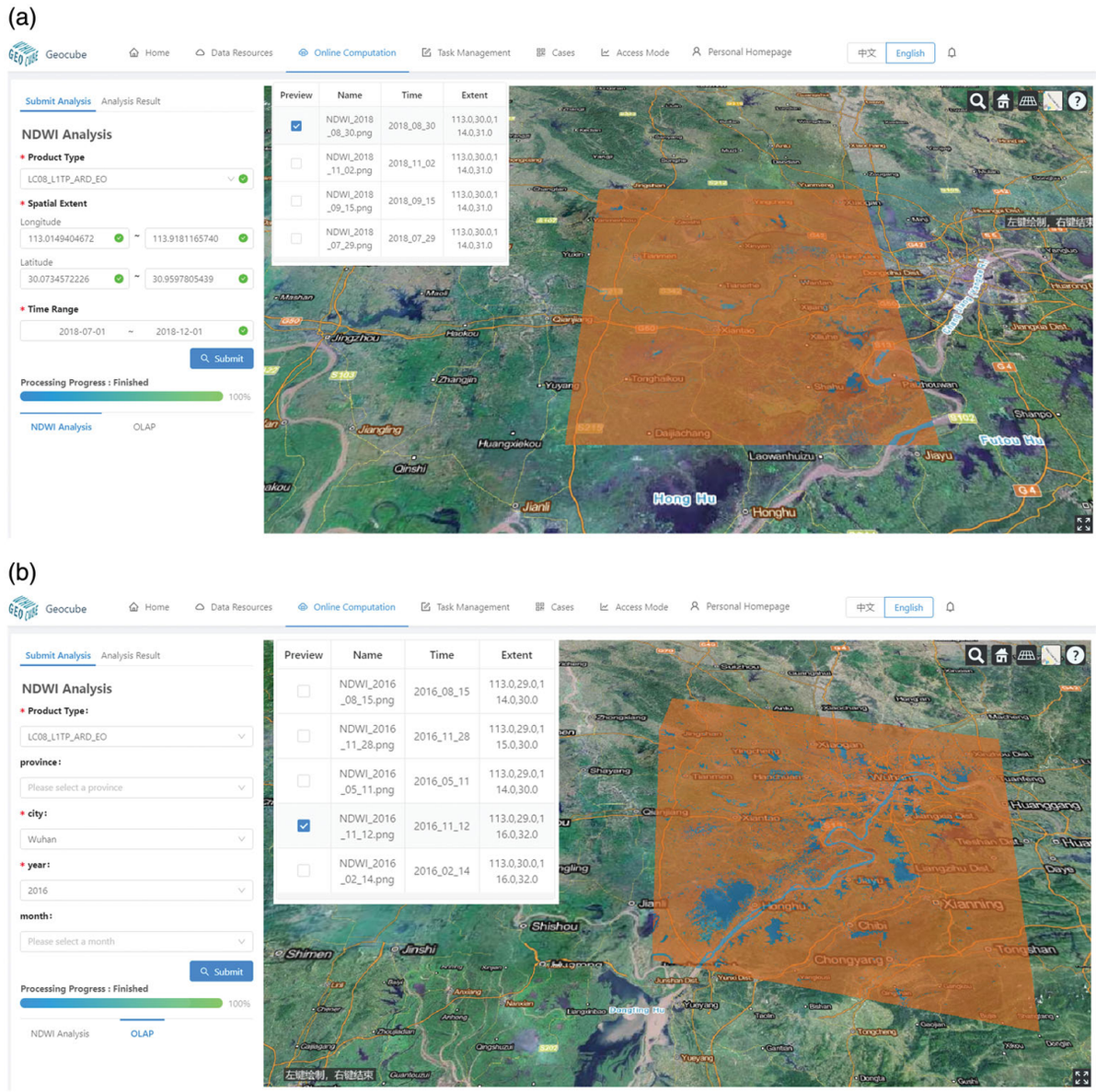


Figure 8. OLAP operations in GeoCube.

Figure 9 presents the results of the water presence frequency analysis, which are derived from Landsat 8 images covering Hubei province from 2013 to 2018. High values in the results indicate that the region is detected as the water body most of the time, and vice versa. The analysis considers the benefits of GeoCube for batch processing, cloud computing, and multi-thread computing. The tiles are divided into batches along the spatial dimension to ensure a full time-series stack of tiles. Approximately 1 TB of data are used in the analysis. Approximately 1.4 h are required to load the data into memory and 1 h is required to complete the processing. A more comprehensive performance analysis is provided in Section 4.3.2.

4.2.3. Multi-source data cube analysis

GeoCube differs from existing cube approaches in that it can support multi-source data analysis in cubes. This section presents a flood disaster case as an example to demonstrate how multi-source data are used in GeoCube for a rapid response. It is assumed that the disaster was caused by a tropical cyclone striking southern China

SNOW MELT IN SLIM BUTTES

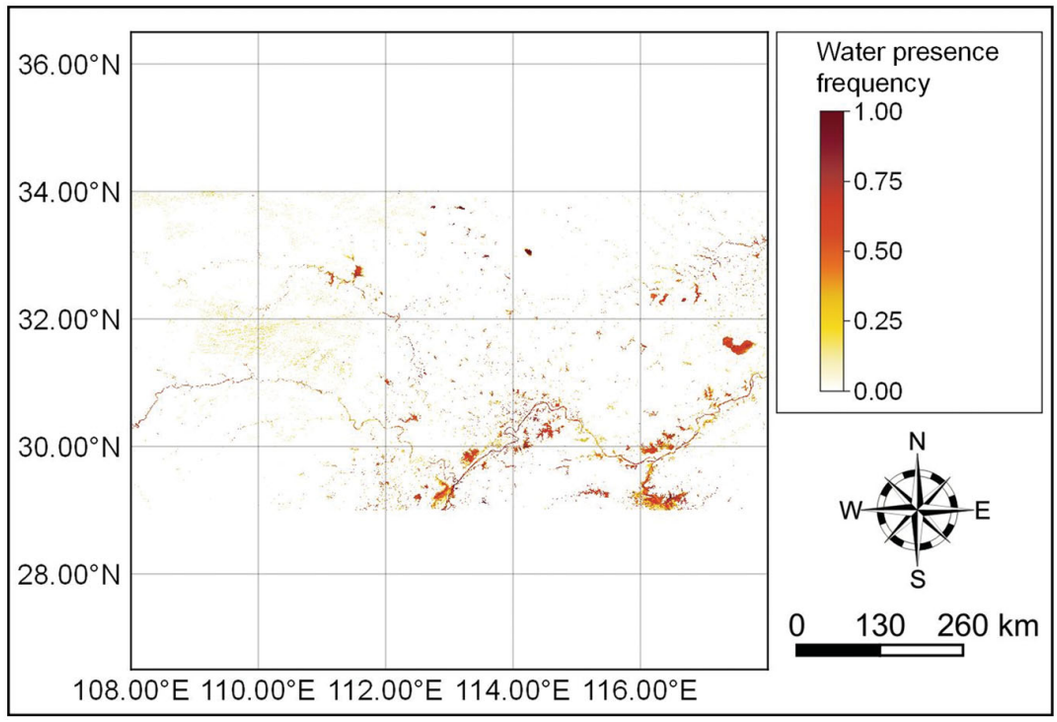


Figure 9. Product of water presence frequency analysis using Landsat-8 data covering Hubei province from 2013 to 2018.

and northern Vietnam. The hurricane was accompanied by heavy rain, resulting in flood disasters in cities. Thousands of people were evacuated and many facilities were impacted. This case involves raster data (e.g. remote sensing images), vector data (e.g. facility features), and tabular data (e.g. populations). GeoCube can help to derive flood inundation data products, impacted facilities, and affected populations using analytics of these multi-source data.

In the case implementation, the water bodies before and after the disaster are extracted from Gaofen-1 images, following which the flood inundation product can be generated by subtracting the pre- and post-disaster data. Subsequently, impacted facilities such as roads, bridges, villages, and schools, can be extracted using the flood inundation product. Figure 10 depicts the flood inundation product (red region) and impacted bridges (red icons). Furthermore, the population data can be used to derive the affected populations by matching the village name with the vector data. Figure 11 presents the process of extracting the impacted villages and affected populations during the flood disaster using Jupyter Notebook. The process includes importing the GeoCube kernel, multi-dimensional querying, evaluating the impacted bridges and affected populations, and displaying the results. The kernel contains the core APIs of the GeoCube, such as *getData()*. All of these data products and statistical results can quickly provide informational support for making disaster response decisions.

QUERING THE CUBE!!

4.3. Performance evaluation

This section presents experiments to demonstrate the performance of GeoCube.

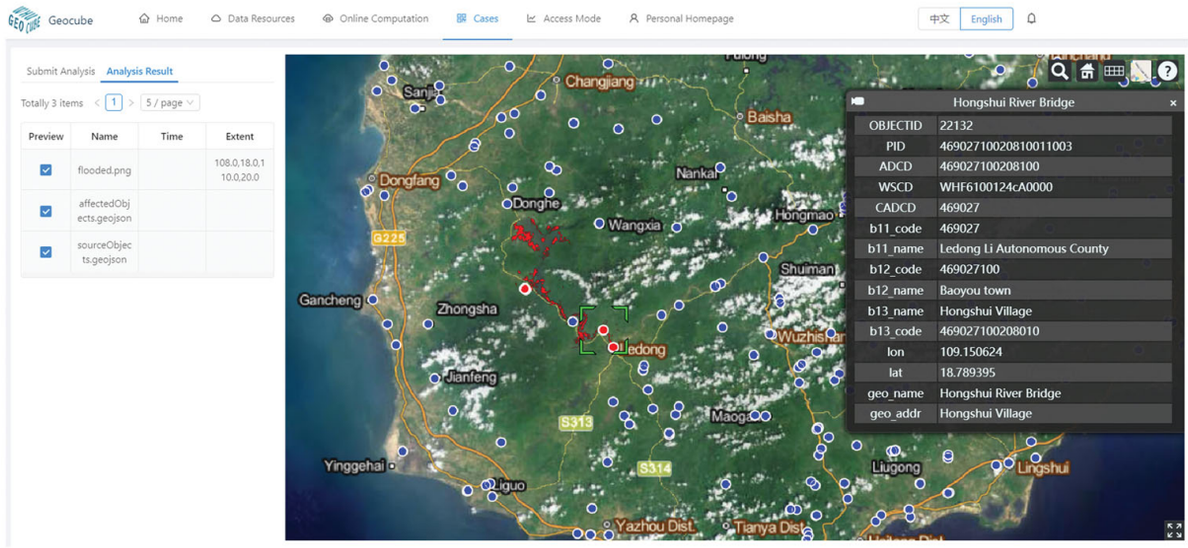


Figure 10. Flood inundation product (red region) and impacted bridges (red icons) in flood disaster.

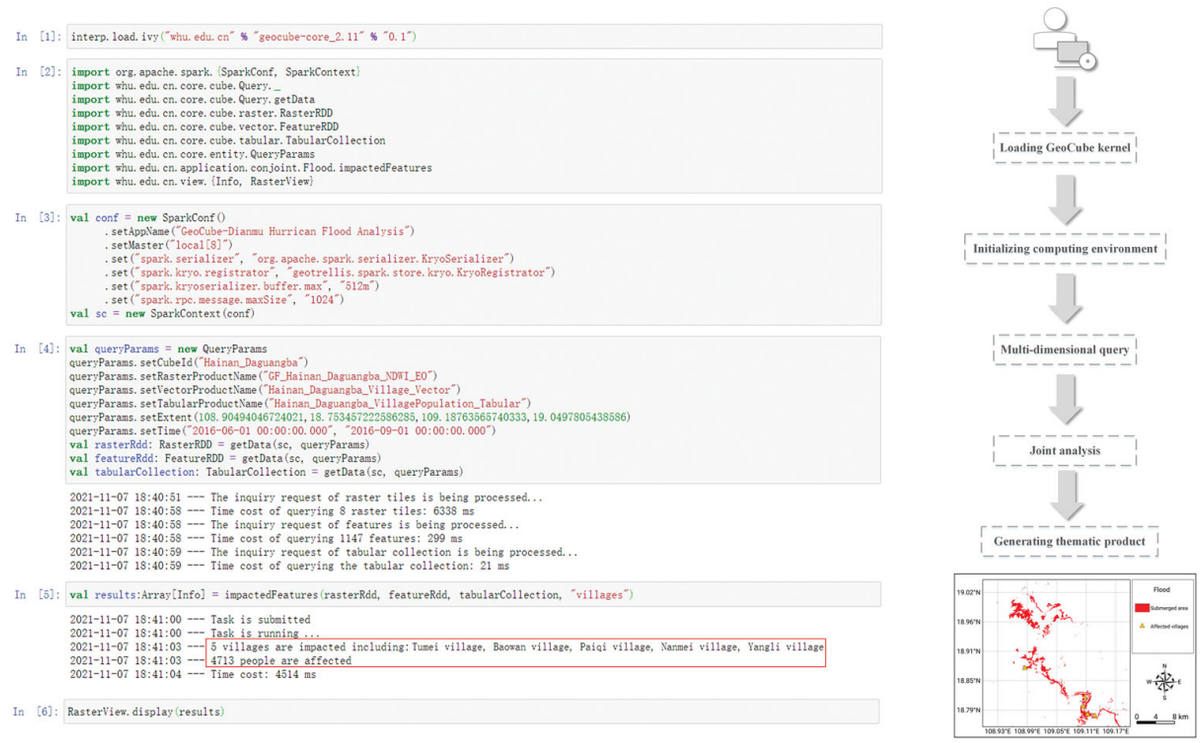


Figure 11. Process of extracting impacted villages and affected populations in flood disaster using Jupyter Notebook.

4.3.1. Data and environment

In the experiments, the raster data in GeoCube included data from the Landsat-8 Operational Land Imager sensor with a 30 m resolution, Gaofen 1 Panchromatic & Multispectral Scanners with a resolution of 2/8 m, and Wide-Field-of-View sensor with a resolution of 16 m. The vector data in the GeoCube were acquired from OSM.

GeoCube was deployed on three physical servers. Two were Inspure NF5280M4 servers, each with an Intel(R) Xeon(R) CPU E5-2690 v4 @ 2.60 GHz, CentOS Linux release 7.3.1611 OS, 28 CPU cores, and 128 GB of RAM. The other was an Inspure

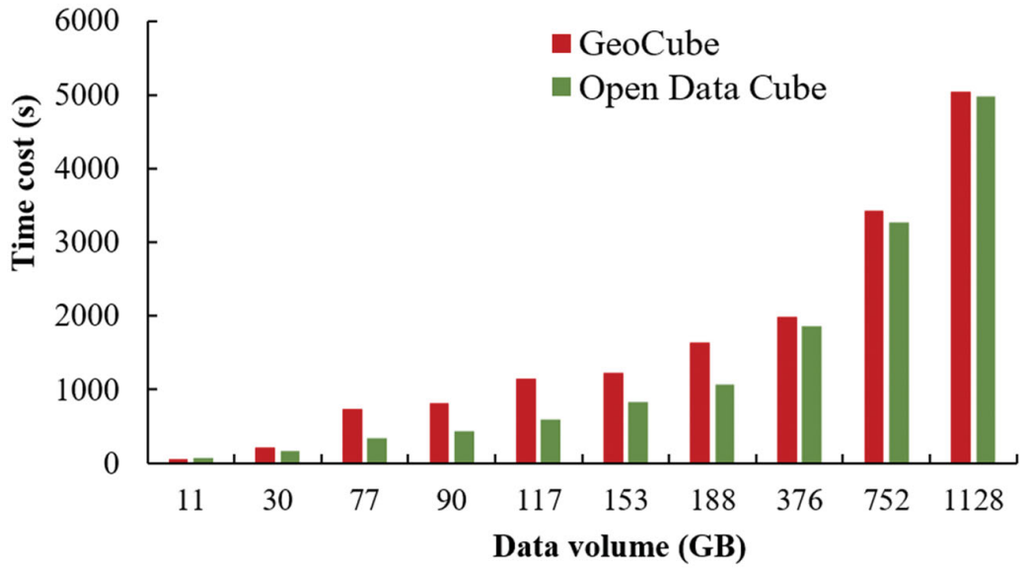


Figure 12. Data query efficiency: GeoCube vs. Open Data Cube.

NF5270M3 server with an Intel(R) Xeon(R) CPU E5-2692 v2 @ 2.20 GHz, Ubuntu 14.04.1 OS, 24 CPU cores, and 94 GB of RAM. These servers were connected to a high-performance storage array with PB-level storage space. Specifically, 84 storage disks were included, each of which had 16 TB of data storage.

CYVERSE

4.3.2. Query and processing performance

In this experiment, the water presence frequency analysis was again used as an example, and the performances of the ODC and GeoCube were compared to demonstrate the query and processing capability of GeoCube. The ODC and GeoCube were deployed in the same environment. ODC 1.8.3 was used, and the image data were ingested as tiles and saved in the file system. The Xarray 0.18.0 and Dask 2021.3.0 libraries were used for the distributed computing.

Figure 12 presents the data query performances of the ODC and GeoCube, which involved tile ID retrieval and tile data loading. It can be observed that the ODC exhibited better performance than GeoCube when the data volume was small. However, as the data volume increased, the time query efficiency of GeoCube was almost same as that of ODC. The reason for this phenomenon is that the data are saved in a distributed environment in GeoCube. Compared to the ODC, in which the data are saved centrally in a local file system, more network communications occur in the querying process in GeoCube because the queried data must be obtained from other nodes if they are not saved in the current node.

The distributed storage approach had a certain negative effect on the query efficiency, however, it could improve the computing efficiency significantly because the data already resided in the distributed nodes. Figure 13 presents the processing performances of the ODC and GeoCube. It can be observed that GeoCube outperformed the ODC, regardless of the data volume. GeoCube exhibited a greater time saving than the ODC as the data volume increased. There are two reasons for this phenomenon: first, the data resided in the distributed nodes in advance, thereby reducing the communication cost during processing, and second, the proposed hybrid parallel

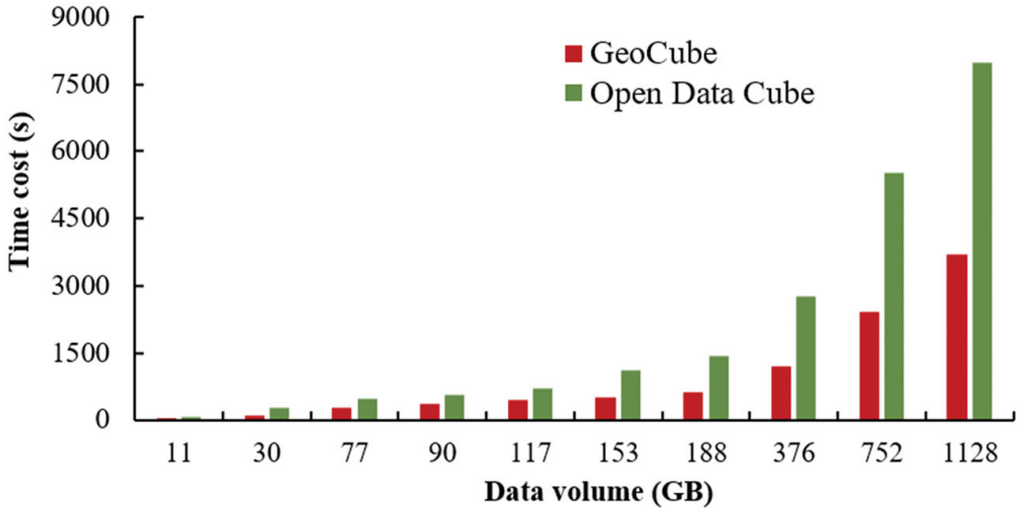


Figure 13. Data processing efficiency: GeoCube vs. Open Data Cube.

computing approach, including the multiple processes in cloud computing and multiple threads, significantly improved the processing efficiency.

4.3.3. Load balance performance

The spatial intersection of vector data is used as an example to demonstrate the load balance performance of GeoCube. The original parallel spatial intersection method without AI augmentation in GeoCube is a classical method, which is known as the partition-based spatial-merge join (Patel and DeWitt 1996). The data were decomposed based on a regular grid, which ignored the heterogeneity of the data, thereby leading to a serious load imbalance.

This section presents a comparison of the load balance performance of the original approach and the AI-enabled CI prediction approach in GeoCube. The vector data were collected from OSM covering China and India. Three tests were implemented: the dataset in the first test contained two layers with 1.69 and 0.49 million polygons, the dataset in the second test contained two layers with 2.50 and 0.82 million polygons, and the dataset in the third test contained two layers with 3.00 and 0.87 million polygons. The CI of each vector tile is pre-computed using a CI prediction model and saved in a JSON file in the AI approach, whereas the CI is ignored in the original approach.

Figure 14 presents the accuracy of different CI prediction models based on the combination of feature selection and machine learning algorithms. The training sample contained approximately 1200 records, covering various areas of the datasets. Four feature selection algorithms, namely Pearson, recursive feature elimination (RFE), mean decrease in impurity (MDI) and mean decrease in accuracy (MDA) are tested together with all features (AF), on four machine learning algorithms, namely classification and regression tree (CART), random forest (RF), gradient boosting regression (GBR) and support vector regression (SVR) (Yue *et al*. 2020). The two metrics R2 and mean absolute error (MAE) were used to evaluate the model accuracy under 10-fold cross-validation. The results revealed that the model using a combination of MDI and RF performed the best. The R2 and MAE of the model were 0.78 and 456 ms, respectively.

Figure 15 presents the time costs of different stages using the original approach and AI-enabled CI prediction approach. With an increase in the number of polygons,

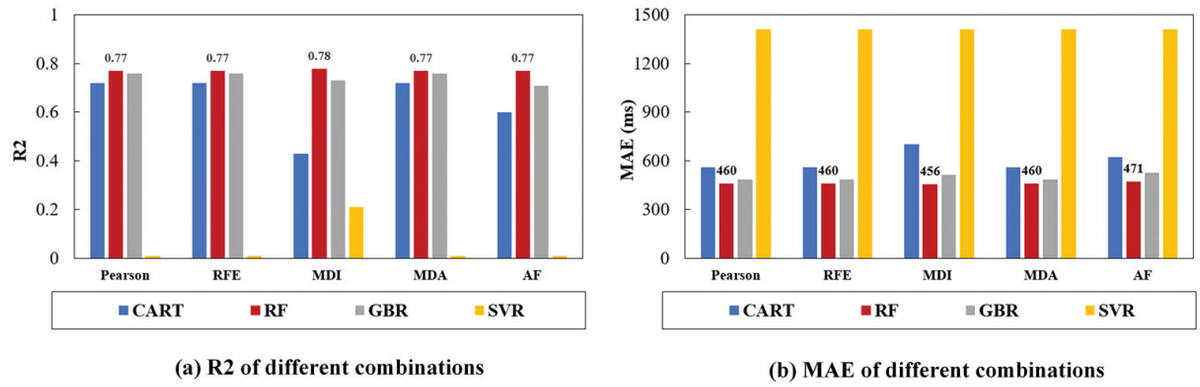


Figure 14. Accuracy of different models from combinations of feature selection and machine learning algorithms: (a) R2 and (b) MAE.

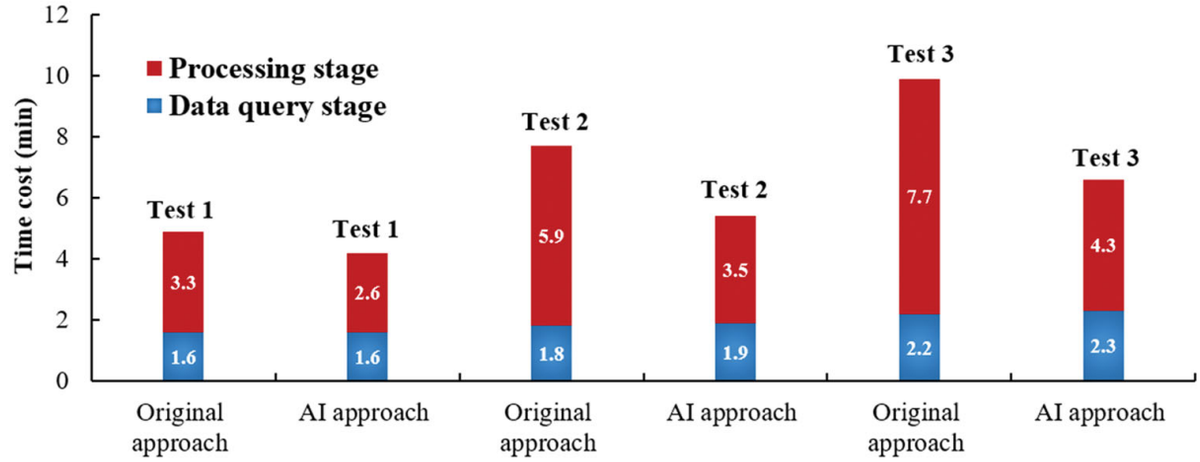


Figure 15. Computing efficiency comparison between original and AI approaches.

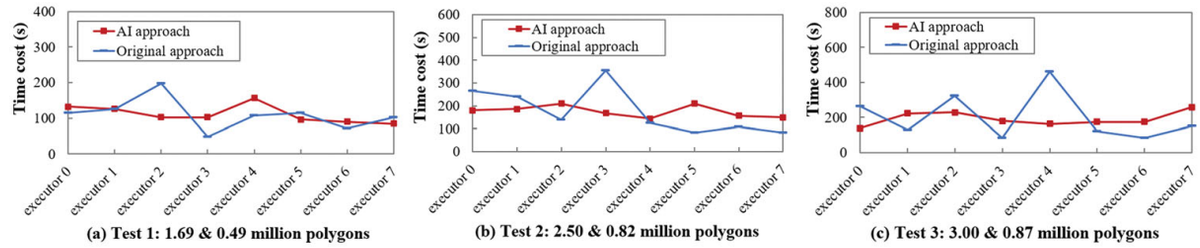


Figure 16. Load balance performance comparison between original and AI approaches.

the time cost of the data query stage was nearly equal when using the two approaches. However, the AI approach exhibited a greater time saving compared to the original approach in the processing stage. The load balance performance of the two approaches on the three datasets is presented in Figure 16, which indicates the time cost for each executor. The results demonstrate that the AI approach could achieve a better load balance performance than the original approach.

5. Discussions

Both EO and BI data cubes have been advocated in big data analytics, and it is important for the two to be harmonized to support OLAP operations. The significant

difference between EO and BI data cubes is that the pixels in EO data cubes are very dense, whereas the elements in BI data cubes are always sparse, which may limit the use of the BI OLAP approach for EO data cubes. Although both cubes can define the dimensions and facts/cells, a key challenge is the management of densely populated pixels in a high-performance form. The high-performance form should be capable of dealing with EO images, as well as accommodating multi-source geospatial data by considering their spatial heterogeneity and different data structures. To address these issues, this study has presented an augmented cube with multi-source geospatial data ingestion and management as well as the processing of multiple source data in a cloud computing environment. Such a cube can inherit the large-scale processing capabilities of EO data cubes and the data management capabilities of BI data cubes.

Several dimensions are customized for the raster and vector data in the cube modeling. However, this model is not limited to raster and vector data, it can be extended to accommodate more data sources, such as trajectory data, by incorporating fact and dimension tables. This capability is borrowed from BI data cubes. Furthermore, this approach enables OLAP/SOLAP operations on the cube data, which can be performed on the fly or using the precomputing mode. The latter computes several commonly used products in advance so that storage can be traded in for a rapid response to users (Han *et al*. 1998, Koperski *et al*. 1998).

The integration of multi-source geospatial data is performed under the premise of the cell granularity and shared formalization of the dimensions for the multi-source data. As opposed to existing EO cubes, the proposed method introduces the idea of SOLAP in traditional BI cubes by adding the spatial measure. Thus, the cell is not the pixel of existing EO cubes, but may be a group of pixels and a set of spatial entities in terms of the raster and vector, respectively. Furthermore, it is assumed that the raster/vector/tabular data can have the same spatial dimension in a spatially aligned cube. Although this requirement is sufficient for raster and vector data, it is not often the case for tabular data. In the experiment that was conducted in this study, we assumed that the population tabular data had place names, which could be geocoded and transformed into the spatial dimension. If this is not the case, it is possible to create a BI cube for the tabular data and to set up a common dimension based on the common attributes in the vector/tabular data. Thus, integration may still occur based on the joint connection in a common dimension instead of the spatial dimension.

The data cube requires the re-organization of external data into a high-performance form. This involves the duplication of data during the data ingestion, which will increase the overhead. Typically, the overhead includes the storage overhead as well as the potential re-sampling and re-projection overhead for spatial alignment. Thus, it will be valuable to explore the optimization of the duplication, such as a zipped form, distributed files, or NoSQL solutions. Parallel data ingestion is also possible, for example, in a cloud environment, the rationale for which is the same as that of OLAP in terms of trading in storage for rapid responses.

Cloud computing has been used extensively in geospatial infrastructure. It can significantly improve the data storage and processing capabilities of cubes. Although the cloud may lead to other issues such as privacy and accessibility, cube vendors can opt to deploy the private cloud on their own hardware using open solutions, thereby

maintaining the cube data accessibility in a private environment. Moreover, the coupling of the cubes and external data is flexible: external data vendors can deploy the cube system to provide cube access or data can be downloaded from external data vendors and ingested into local cubes. Although GeoCube is currently deployed in a private cloud environment using local computing clusters, it may be used in public cloud environments. The proposed distributed cube objects based on the Spark middleware can be deployed on any virtual machine in a public infrastructure-as-a-service (IaaS) environment. The use of AI methods to optimize the load balance in the computation may be effective in capturing the heterogeneous computational features of the geoprocessing. However, such method is more suitable for commonly used products with fine-tuned prediction models. If the training cost for the AI evaluation of the CI in specific analytical functions is excessive, straightforward tile dissemination based on tile numbers may be preferable in simple computations, such as map algebra operation, compared to complex ones, such as vector tile analysis.

GeoCube demonstrates the applicability of the proposed methods on Landsat, Gaofen, Sentinel, and OSM. Other EO data sources are currently being tested and supported, and future work will extend the system to provincial EO data centers. A performance comparison between GeoCube and the ODC has been presented, and GeoCube outperformed the ODC in terms of data processing efficiency. In the experiment, the ODC used the NetCDF format for data storage, which is known for its limited capabilities in the cloud environment. Cloud Optimized GeoTIFF (COG) is an alternative imagery data format for the cloud environment (COG 2021), and ODC users are attempting to use this format to improve the performance. A comparison of the ODC based on COG with GeoCube could form an aspect of our future work.

The alignment of multi-source data with different spatial and temporal resolutions is a challenge, which often requires re-sampling, re-projection, and various preprocessing steps such as radiometric and geometric correction. Therefore, ARD is suggested as the input for the EO data cube. ARD is currently a popular term in the EO domain for satellite imagery data products. It is commonly defined as a type of products following a series of preprocessing operations (Killough *et al*. 2020). As a multi-source geospatial cube infrastructure, GeoCube contains satellite imagery data, as well as vector and tabular data. At present, no definition for vector and tabular ARD is available. Based on the experience in this work, it is possible to suggest a definition of ARD from the data production and computational perspectives. Regarding data production, vector and tabular ARD mean that the data are quality assured at different levels in terms of completeness, consistency, and accuracy. From a computational perspective, vector and tabular ARD need to be organized in a high-performance ready and spatially aligned form, for example, tiling and using formats such as COG.

The GeoCube kernel API remains an ad-hoc interface (e.g. QueryParams), and interoperability is an issue in data cube communities. The OGC Testbed-17 is currently exploring the identification of a neutral definition of Geo Data Cubes (GDCs) and the development of a Web API for convenient, interoperable GDC access and exploitation (OGC 2021). Moreover, CEOS has suggested the use of the ODC in various countries, such as Colombia and Switzerland, with the aim of alleviating the interoperability issue (CEOS 2021). Finally, the OGC coverage model can be used to represent a multi-

dimensional data cube, and the WCS can support cube operations such as slicing and trimming (Baumann 2018). The data cube is still under development and we are exploring new ideas to improve its interoperability.

6. Conclusions and future work

This study has presented a cube infrastructure for big geospatial data analysis. The infrastructure accommodates multi-source geospatial data, including raster and vector data. A cube data model was designed and four dimensions, namely product, space, time, and band dimensions, were formalized. Several examples of cube exploration have been presented. The infrastructure enables large-scale analysis using a hybrid parallel method. A set of distributed cube objects extending the Spark RDD for cube tiles was designed. The distributed cube objects are compatible with multiple data sources, including raster and vector data. To improve the data processing efficiency further, a multi-thread computing method was employed together with cloud computing to form hybrid parallelism. Batch computation was also used to address the issue whereby a massive number of tiles cannot be loaded into memory simultaneously. Moreover, a machine learning-based approach was integrated into the cube to enhance the parallel geoprocessing. The CI of the tiles can be predicted and saved in databases in advance, thereby eliminating the additional time cost of on-the-fly CI prediction for commonly used products.

The design and implementation of the cube infrastructure, which is known as GeoCube, have been outlined. It covers the ingestion and management of multi-source geospatial data in the cube, geospatial data querying and access along different cube dimensions, and high-performance cube computing of large-scale geospatial datasets. The development of such a geospatial data cube aids in advancing the EO data cube approach as well as keeps connections with the BI cube. The performance of the data querying and access, data processing, and load balancing were presented. The results demonstrate the advantages of the GeoCube infrastructure. Several applications have been presented, including cube OLAP operations, large-scale time-series analysis, and multi-source data cube analysis. The results on Landsat, Gaofen, and OSM demonstrate the applicability of GeoCube.

The data cube infrastructure is still under development and receiving increasing attention owing to its potential for EO data management and processing. In addition to interoperability, the following issues should be investigated in the future: (1) Discrete global grid systems (DGGS) exhibit promise as processing-efficient tiling grids for both raster and vector data (Purss *et al*. 2019). It would be worthwhile to investigate this tiling mechanism for more efficient cube processing. (2) In this study, the AI was limited to improving the cube processing performance. Work has been carried out on the use of AI to optimize data index, such as learned index (Kraska *et al*. 2018). This can be explored in the future to optimize the data index of the cube. (3) The current cube analysis remains limited to traditional analytical functions. Deep learning analytics has been investigated extensively in the geospatial domain. It is necessary to explore the manner in which deep learning frameworks can be incorporated into the cube infrastructure to enable faster model training and data inference based on the

cube capabilities. Thus, intelligent interpretation models for object detection, scene classification, and change detection can be provided in a well-organized cube infrastructure. (4) ARD is the primary data form for traditional analysis in the current cube, and AI training data are another type of data form for deep learning. Thus, it is valuable to explore a new AI-ready data form to promote the cube infrastructure. In addition to the two types of ready data, decision-ready data that are used by decision makers are also important, which are often derived by a workflow involving ARD, AI-ready data, and analysis models. For example, decision makers may require supporting information regarding a flooded road that is only passable by special vehicles. Such information can be derived as decision-ready data using ARD such as terrain topography and flood gauges. Decision-ready data can also be integrated into the current cube to form a three-ready-data (i.e. ARD, AI-ready data, and decision-ready data) geospatial infrastructure. Such a cube infrastructure will promote the development of the current paradigm (i.e. ARD processing) in the EO domain.

Acknowledgements

We appreciate Professor Jennifer Miller and three reviewers for their constructive comments that helped improve the quality of the paper. The work was supported by National Natural Science Foundation of China (No. 42090011 and No. 42071354), and Major Science and Technology Projects of Hubei Province (No. 2020AAA004).

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

The work was supported by National Natural Science Foundation of China [No. 42090011 and No. 42071354], and Key Research and Development Program of Hubei, China [No. 2020AAA004].

Notes on contributors

Fan Gao is a Ph.D. student in the School of Remote Sensing and Information Engineering at Wuhan University. His research interest is in high-performance geoprocessing and GIS. He contributed to the idea, study design, methodology, implementation, and manuscript writing of this paper.

Peng Yue is a professor at Wuhan University. He serves as the director at Hubei Province Engineering Center for Intelligent Geoprocessing and the director at the Institute of Geospatial Information and Location Based Services. He supervised the research, and contributed to the idea, study design, methodology, and manuscript writing of this paper.

Zhipeng Cao is a Ph.D. student in the School of Remote Sensing and Information Engineering at Wuhan University. His research interest is in big spatio-temporal data stores. He contributed to the idea, methodology, and implementation.

Shuaifeng Zhao is a Ph.D. student in the School of Remote Sensing and Information Engineering at Wuhan University. His research interest is in big spatio-temporal data stores and processing. He contributed to the idea, methodology, and implementation.

Boyi Shangguan is a Ph.D. student in the School of Remote Sensing and Information Engineering at Wuhan University. His research interest is in high-performance geoprocessing and GIService. He contributed to the idea, methodology, and implementation.

Liangcun Jiang is an associate research fellow in the School of Remote Sensing and Information Engineering at Wuhan University. His research interests include Earth observation data infrastructure, data provenance, and geospatial semantic web. He contributed to the idea, study design, and methodology.

Lei Hu is a postdoctoral researcher in the School of Remote Sensing and Information Engineering at Wuhan University. His research interests include Earth observation science and systems, geospatial interoperability and standards, and environmental modelling in agriculture and disaster management. He contributed to the idea, study design, and methodology.

Zhe Fang is a Ph.D. student in the School of Remote Sensing and Information Engineering at Wuhan University. His research interest is in disaster emergency response. He contributed to the idea, methodology, and implementation.

Zheheng Liang is the chief engineer of South Digital Technology Co., Ltd. He currently is the special reviewer of the journal 'Geographic Information World', and an expert in the comprehensive evaluation expert database of Guangdong Province. He contributed to the idea, study design, and methodology.

ORCID

Peng Yue http://orcid.org/0000-0003-3006-4542

Data and codes availability statement

The data and codes that support the findings of this study are available in https://doi.org/10.6084/m9.figshare.15032847.

References

Baumann, P., 2017. *The datacube manifesto* [online]. EU EarthServer. Available from: https://earthserver.eu/tech/datacube-manifesto/The-Datacube-Manifesto.pdf [Accessed 22 July 2021].

Baumann, P., 2018. Datacube standards and their contribution to analysis-ready data. In*: 2018 IEEE international geoscience and remote sensing symposium (IGARSS), 22–27 July 2018* Valencia. New York: IEEE, 2051–2053.

Baumann, P., *et al.*, 1999. Spatio-temporal retrieval with RasDaMan. In*: Proceedings of 25th international conference on very large data bases, 7–10 September 1999* Edinburgh. Burlington: Morgan Kaufmann, 746–749.

Baumann, P., *et al.*, 2016. Big data analytics for earth sciences: the EarthServer approach. *International Journal of Digital Earth*, 9 (1), 3–29.

Baumann, P., *et al.*, 2018., Fostering Cross-Disciplinary Earth Science Through Datacube Analytics. In*:* P.P. Mathieu and C. Aubrecht, eds. *Earth observation open science and innovation*. Switzerland: Springer Cham, 91–199.

Berrick, S., *et al.*, 2009. Giovanni: a web service workflow-based data visualization and analysis system. *IEEE Transactions on Geoscience and Remote Sensing*, 47 (1), 106–113.

Bimonte, S., *et al.*, 2010. When spatial analysis meets OLAP: multidimensional model and operators. *International Journal of Data Warehousing and Mining*, 6 (4), 33–60.

CEOS, 2021. *Open Data Cube* [online]. CEOS. Available from: https://www.opendatacube.org/ceos [Accessed 22 July 2021].

Cesium, 2021. *The Platform for 3D Geospatial* [online]. Cesium GS, Inc. Available from: https://cesium.com [Accessed 22 July 2021].

Chaudhuri, S., and Dayal, U., 1997. An overview of data warehousing and OLAP technology. *ACM SIGMOD Record*, 26 (1), 65–74.

COG, 2021. *Cloud Optimized Geotiff* [online]. Cloud Optimized GeoTIFF community. Available from: https://www.cogeo.org [Accessed 26 October 2021].

Damiani, M.L., and Spaccapietra, S., 2008. Spatial data warehouse modelling. *In:* J. Wang, ed. *Data warehousing and mining: concepts, methodologies, tools, and applications*. Hershey: IGI Global, 659–678.

Dask, 2021. *Scalable analytics in Python* [online]. NumFOCUS community. Available from: https://dask.org [Accessed 22 July 2021].

ESA, 2012. *Grid processing on demand (G-POD) for earth observation applications* [online]. European Space Agency (ESA). Available from: https://gpod.eo.esa.int [Accessed 26 October 2021].

ESDC, 2021. *Earth system data cube* [online]. The Earth System Data Cube (ESDC) Team. Available from: https://cablab.readthedocs.io/en/latest [Accessed 22 July 2021].

George, L., 2011. *HBase: the definitive guide: random access to your planet-size data*. California: O'Reilly Media Inc.

Geotrellis, . 2021. *A geographic data processing engine for high performance applications* [online]. Azavea Inc. Available from: https://geotrellis.readthedocs.io/en/latest [Accessed 22 July 2021].

Giuliani, G., *et al.*, 2017. Building an earth observations data cube: lessons learned from the Swiss Data Cube (SDC) on generating analysis ready data (ARD). *Big Earth Data.*, 1 (1–2), 100–117.

Gomes, V.C.F., *et al.*, 2021. Accessing and processing Brazilian earth observation data cubes with the open data cube platform. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, V-4-2021, 153–159.

Gorelick, N., *et al.*, 2017. Google earth engine: planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202, 18–27.

Gray, J., *et al.*, 1997. Data cube: a relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data Mining and Knowledge Discovery*, 1 (1), 29–53.

Han, J., *et al.*, 1998. Selective materialization: An efficient method for spatial data cube construction. In*: Pacific-Asia conference on knowledge discovery and data mining, 15–17 April 1998*. Australia. Berlin: Springer, 144–158.

Jupyter, 2021. *Project Jupyter* [online]. Jupyter community. Available from: https://jupyter.org [Accessed 22 July 2021].

Kasprzyk, J.P., and Devillet, G., 2021. A data cube metamodel for geographic analysis involving heterogeneous dimensions. *ISPRS International Journal of Geo-Information*, 10 (2), 87.

Killough, B., *et al.*, 2020. Advancements in the open data cube and analysis ready data—past, present and future. In*: 2020 IEEE international geoscience and remote sensing symposium (IGARSS)*, 16–26 July 2018. Waikoloa Village, New York: IEEE, 3373–3375.

Kimball, R., and Ross, M., 2011. *The data warehouse toolkit: the complete guide to dimensional modeling*. New York: John Wiley & Sons Inc.

Koperski, K., *et al.*, 1998. Mining knowledge in geographical data. *Communications of the ACM*, 26 (1), 65–74.

Kopp, S., *et al.*, 2019. Achieving the full vision of earth observation data cubes. *Data*, 4 (3), 94.

Kraska, T., *et al.*, 2018. The case for learned index structures. *In*: *Proceedings of the 2018 international conference on management of data, 10–15 June 2018*. Houston, New York: ACM Digital Library, 489–504.

Lewis, A., *et al.*, 2016. Rapid, high-resolution detection of environmental change over continental scales from satellite data–the earth observation data cube. *International Journal of Digital Earth*, 9 (1), 106–111.

Lewis, A., *et al.*, 2017. The Australian geoscience data cube—foundations and lessons learned. *Remote Sensing of Environment*, 202 (1), 276–292.

Lewis, A., *et al.*, 2018. CEOS analysis ready data for land (CARD4L) Overview. *In*: *IEEE international geoscience and remote sensing symposium (IGARSS), 22–27 July 2018*. Valencia, New York: IEEE, 7407–7410.

LLNL, 2021. *Introduction to Parallel Computing Tutorial* [online]. Lawrence Livermore National Laboratory (LLNL). Available from: https://hpc.llnl.gov/training/tutorials/introduction-parallel-computing-tutorial [Accessed 22 July 2021]

Mahecha, M.D., *et al.*, 2020. Earth system data cubes unravel global multivariate dynamics. *Earth System Dynamics*, 11 (1), 201–234.

Momjian, B., 2001. *PostgreSQL: introduction and concepts*. New York: Addison-Wesley.

Mueller, N., *et al.*, 2016. Water observations from space: mapping surface water from 25 years of Landsat imagery across Australia. *Remote Sensing of Environment*, 174 (1), 341–352.

Nativi, S., *et al.*, 2017. A view-based model of data-cube to support big earth data systems interoperability. *Big Earth Data.*, 1 (1–2), 75–99.

OGC, 2021. *Testbed-17. OGC testbed initiative* [online]. OGC. Available from: https://portal.ogc.org/files/?artifact_id=95726 [Accessed 22 July 2021].

Patel, J.M., and DeWitt, D.J., 1996. Partition based spatial-merge join. *ACM SIGMOD Record*, 25 (2), 259–270.

Pedregosa, F., *et al.*, 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12 (10), 2825–2830.

Purss, M.B.J., *et al.*, 2019. Datacubes: a discrete global grid systems perspective. *Cartographica*, 54 (1), 63–71.

Rasdaman, 2021. *Raster data manager* [online]. Rasdaman GmbH. Available from: http://rasdaman.com [Accessed 22 July 2021].

Rivest, S., *et al.*, 2001. Toward better support for spatial decision making: defining the characteristics of spatial on-line analytical processing (SOLAP). *Geomatica*, 55 (4), 539–555.

Schramm, M., *et al.*, 2021. The openeo api–harmonising the use of earth observation cloud services using virtual data cube functionalities. *Remote Sensing*, 13 (6), 1125.

Silva, J.D., *et al.*, 2008. A set of aggregation functions for spatial measures. In*: Proceedings of the ACM 11th international workshop on Data warehousing and OLAP, 30 October*. Napa Valley, New York: ACM Digital Library, 25–32.

Spring Boot, 2021. *Spring* [online]. VMware, Inc. Available from: https://spring.io/projects/spring-boot [Accessed 22 July 2021].

Strobl, P., *et al.*, 2017. The six faces of the data cube. In*: Proceedings of big data from space conference, 28–30 November 2017*. Toulouse. Luxembourg: Publications Office of the European Union, 28–30.

Sudmanns, M., *et al.*, 2019. Big Earth data: disruptive changes in earth observation data management and analysis? *International Journal of Digital Earth*, 13 (7), 832–850.

Voidrot, M.F., and Percivall, G., 2020. OGC geospatial coverages data cube community practice. Vol. 509. In: *11th International symposium on digital earth, 24–27 September 2019*. Florence. Bristol: IOP Publishing, 012058.

Wagemann, J., *et al.*, 2018. Geospatial web services pave new ways for server-based on-demand access and processing of big earth data. *International Journal of Digital Earth*, 11 (1), 7–25.

White, T., 2012. *Hadoop: The definitive guide*. California: O'Reilly Media Inc.

Wulder, M.A., and Coops, N.C., 2014. Satellites: make earth observations open access. *Nature*, 513 (7516), 30–31.

Xarray, 2021. *N-D labeled arrays and datasets in Python* [online]. NumFOCUS. Available from: http://xarray.pydata.org/en/stable [Accessed 22 July 2021].

Yue, P., *et al.*, 2010. GeoPW: laying blocks for the geospatial processing web. *Transactions in GIS*, 14 (6), 755–772.

Yue, P., *et al.*, 2016. Recent activities in earth data science. *IEEE Geoscience and Remote Sensing Magazine*, 4 (4), 84–89.

Yue, P., *et al.*, 2020. A machine learning approach for predicting computational intensity and domain decomposition in parallel geoprocessing. *International Journal of Geographical Information Science*, 34 (11), 2243–2274.

Zaharia, M., *et al*., 2010. *Spark: Cluster computing with working sets*. *In: Proceedings of the 2nd USENIX conference on Hot topics in cloud computing*, 22–25 June 2010, Boston. California: USENIX Association, 10.

Zarr. 2021. *Zarr format* [online]. MRC Centre for Genomics and Global Health. Available from: https://zarr.readthedocs.io [Accessed 22 July 2021].

Zhao, P., *et al*., 2012. The geoprocessing web. *Computers & Geosciences*, 47 (10), 3–12.

Copyright of International Journal of Geographical Information Science is the property of Taylor & Francis Ltd and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.